SYSTEM DESIGN SCENARIO DRILLS

Read only the scenario and interview question. Speak for 60-90 seconds, then compare the answer. Mark the three boxes after a first correct answer, a next-day recall, and a seven-day recall. Prefer evidence and containment before confident guesses.

1. Production Incidents and Latency

S001 Latency after a deployment [ ] [ ] [ ]

Situation: Your team deployed a new Spring Boot version, and 20 minutes later production latency increased sharply. You suspect the deployment, but you are not completely sure.

"What do you think is happening, and what should we do next?"

Answer: The timing makes a release regression the leading hypothesis, but I would verify correlation before declaring causation. I would compare pre- and post-deployment latency, error rate, CPU, memory, garbage collection, database time, and downstream calls, and split metrics by old and new instance version if both are running. If the new version is clearly worse and users are affected, I would pause the rollout or roll back first. After recovery, I would use traces, logs, configuration differences, and code changes to find the regression and add a canary or automated rollback guard.

S002 Peak-hour response time spike [ ] [ ] [ ]

Situation: A service responds in 200 ms under normal traffic, but response time rises to 3-4 seconds during peak hours.

"How would you investigate this performance issue, and what would you check first?"

Answer: I would first locate where the extra latency is introduced instead of assuming the service only needs more servers. I would compare normal and peak traffic using percentile latency and distributed traces, then check request concurrency, CPU, memory, garbage collection, thread pools, connection pools, database time, cache hit rate, queue depth, and downstream latency. If instances are healthy, I would inspect shared dependencies for slow queries, locks, exhausted connections, retries, or rate limits. The fix follows the bottleneck: scale, optimize queries, tune pools, improve caching, shed load, or change a synchronous dependency.

S003 High p99 but healthy average [ ] [ ] [ ]

Situation: Average latency is 180 ms, but p99 latency is above 5 seconds and users report occasional freezes.

"Why can the average look healthy, and how would you find the tail-latency cause?"

Answer: The average hides a small but important group of very slow requests. I would segment p95 and p99 by endpoint, availability zone, instance, tenant, payload size, cache outcome, and downstream dependency. Traces from slow requests can reveal cold cache misses, large responses, database locks, garbage-collection pauses, connection waits, retries, or one unhealthy shard. I would avoid optimizing the common fast path until I know what distinguishes the tail. The immediate mitigation may be a timeout, concurrency limit, or unhealthy-instance removal; the permanent fix targets the slow cohort and gets its own percentile alert.

S004 CPU is low but latency is high [ ] [ ] [ ]

Situation: Request latency is rising, yet application CPU remains below 35 percent on every instance.

"What could be wrong if CPU is not saturated?"

Answer: Low CPU suggests requests may be waiting rather than computing. I would inspect database and HTTP connection pools, thread-pool queues, lock contention, disk or network I/O, downstream latency, DNS, queue waits, and timeout-driven retries. A blocked thread dump and trace waterfall are more useful here than adding CPU. I would also check whether a concurrency limit is too small or whether a dependency is rate-limiting us. If waiting is confirmed, scaling application instances alone may increase pressure on the shared bottleneck, so I would contain traffic and fix or isolate the constrained dependency.

S005 Error rate rises in only one availability zone [ ] [ ] [ ]

Situation: Overall error rate is moderate, but dashboards show most failures come from one availability zone.

"How would you respond without causing a larger outage?"

Answer: I would treat the zone as a suspected failure domain and compare instance health, network errors, dependency reachability, deployment version, and capacity across zones. If evidence is strong, I would gradually drain traffic from that zone while confirming the remaining zones have enough headroom; an immediate full shift could overload them. I would keep writes and data consistency in mind if stateful components are involved. After stabilization, I would identify whether the cause was infrastructure, configuration drift, or a zonal dependency, then test automatic failover and reserve capacity so future evacuation is safe.

S006 Memory grows until instances restart [ ] [ ] [ ]

Situation: A Java service slowly consumes more memory over several hours and is eventually restarted by the platform.

"How would you distinguish a memory leak from normal caching or load growth?"

Answer: I would compare heap usage after full garbage collections, allocation rate, live-object count, traffic, and cache size over time. A healthy cache should plateau or evict; a leak shows retained memory continuing to grow even after collection and after traffic falls. I would capture heap histograms or dumps near different points and compare dominant object retainers, while checking recent code and configuration changes. To contain impact, I might lower traffic per instance, cap caches, or restart gradually. The fix must remove the retention path and add a long-duration soak test rather than relying on restarts.

S007 Intermittent timeout bursts [ ] [ ] [ ]

Situation: The API is healthy most of the day but experiences two-minute bursts of timeouts every hour.

"What patterns would you investigate?"

Answer: The periodic shape suggests scheduled work or synchronized behavior. I would align timeout timestamps with batch jobs, cache expiry, token refresh, log rotation, backups, autoscaling, certificate checks, garbage collection, and downstream maintenance. Traces and queue or connection-pool metrics should show whether requests wait locally or in a dependency. I would also check for many instances performing the same task at once. If synchronization is the cause, I would add jitter, spread batch work, use stale cache values, or isolate background workloads. I would preserve a timeline so the next recurrence can confirm the hypothesis.

S008 Database latency suddenly doubles [ ] [ ] [ ]

Situation: Application code is unchanged, but database query latency doubles within ten minutes.

"What would you check before scaling the database?"

Answer: I would first identify whether all queries slowed or only a query shape, table, shard, or replica. I would inspect active sessions, locks, connection count, slow-query plans, cache hit ratio, disk latency, replication lag, recent schema or statistics changes, backups, and traffic mix. A changed execution plan or blocking transaction can create the symptom without a capacity shortage. I would cancel or isolate harmful work if safe, route reads away from an unhealthy replica, and protect the database from retry storms. Scaling is appropriate only after evidence shows sustained resource saturation rather than contention or a bad plan.

S009 Service is fast locally but slow in production [ ] [ ] [ ]

Situation: An endpoint takes 80 ms in local testing but more than one second in production.

"How would you explain and investigate the difference?"

Answer: Local tests usually omit production data volume, network hops, contention, security layers, cold caches, and real downstream services. I would reproduce the production request shape and use a distributed trace to separate application time from database, cache, network, and dependency time. I would compare configuration, JVM settings, indexes, payload sizes, and data skew, and avoid using production directly for experiments. A realistic staging load test or replay of sanitized traffic can confirm the bottleneck. The fix should address the production constraint and add a representative performance test so local speed is not mistaken for system speed.

S010 Latency improves after restart [ ] [ ] [ ]

Situation: Restarting the service immediately restores latency, but the problem returns a few hours later.

"What does that tell you, and what would you inspect next?"

Answer: A restart resets process-local state, so I would suspect memory pressure, fragmented pools, leaked connections, stuck threads, unbounded queues, cache growth, or a background task accumulating work. I would capture thread dumps, heap data, pool metrics, and queue depth before restarting whenever customer impact allows. I would correlate degradation with uptime rather than wall-clock time and compare a fresh instance with an old one under the same traffic. Restarting is a containment action, not a fix. The permanent change should bound or release the growing resource and alert on the leading indicator before latency fails.

2. Capacity, Scaling, and Traffic Management

S011 Sudden ten-times traffic increase [ ] [ ] [ ]

Situation: A celebrity shares a post and traffic jumps to ten times normal within two minutes.

"How would you keep the system available?"

Answer: I would protect the critical read path first. Autoscaling can add instances, but it may arrive too slowly and can overload databases, so I would use cached or precomputed content, CDN delivery, request coalescing, rate limits, and load shedding for nonessential features. I would monitor database connections, cache hot keys, queue depth, and dependency quotas while scaling gradually. Writes such as analytics or notifications can be buffered asynchronously. After the event, I would measure the actual bottleneck, increase tested burst capacity, pre-warm likely content, and document which features degrade safely under exceptional demand.

S012 Autoscaling adds servers but latency stays high [ ] [ ] [ ]

Situation: The autoscaler doubles application instances, but p95 latency barely changes.

"Why might horizontal scaling fail to help?"

Answer: More application instances help only when the application tier is the bottleneck and the workload is parallelizable. I would check whether all instances wait on the same database, cache, queue partition, external API, lock, or small connection pool. New instances may even worsen a shared bottleneck by opening more connections or producing more retries. I would use traces and saturation metrics to identify the constrained resource, then cap concurrency and protect it. The long-term fix may be query optimization, partitioning, caching, increasing dependency capacity, or redesigning serialized work rather than continuing to scale stateless servers.

S013 Scale-up or scale-out decision [ ] [ ] [ ]

Situation: A service is approaching CPU and memory limits, and the team debates using larger machines or more machines.

"How would you choose between vertical and horizontal scaling?"

Answer: I would consider workload parallelism, state, failure tolerance, scaling speed, operational complexity, and cost. A larger machine is simple and may help a stateful database or memory-heavy workload, but it has a ceiling and increases the impact of one failure. More machines improve redundancy and elastic capacity when requests can be distributed, but require load balancing, statelessness, coordination, and partition-aware dependencies. I would benchmark both at realistic load and choose the simplest option that meets the growth horizon. Often the near-term answer is a safe vertical increase while preparing horizontal scaling before the next limit.

S014 Uneven traffic across instances [ ] [ ] [ ]

Situation: Two service instances are overloaded while six others are mostly idle.

"What could cause this imbalance?"

Answer: I would inspect load-balancer policy, sticky sessions, long-lived connections, client-side discovery, health checks, and whether traffic is partitioned by a skewed key. Round-robin does not balance work if request cost varies greatly or connections remain pinned. I would compare request count and request cost per instance and check whether unhealthy instances are still advertised. A short-term response may be draining hot instances or disabling unnecessary affinity. Long term, I would use least-loaded or latency-aware balancing where appropriate, distribute hot keys, and ensure autoscaling reacts to per-instance saturation rather than only cluster averages.

S015 One tenant consumes most capacity [ ] [ ] [ ]

Situation: A single enterprise customer generates 60 percent of traffic and degrades service for everyone else.

"How would you protect fairness without unexpectedly breaking that customer?"

Answer: I would measure usage by tenant and introduce explicit quotas, concurrency limits, and weighted fairness rather than a single global limit. The immediate containment could cap the tenant's most expensive operation while preserving essential requests, accompanied by communication if contracts are affected. I would separate high-volume batch work from interactive traffic and consider dedicated capacity for negotiated tiers. Limits should return clear retry guidance and be monitored for false positives. The long-term design aligns capacity and pricing with tenant demand so one workload cannot exhaust shared threads, connections, or database resources.

S016 Read-heavy service is database-bound [ ] [ ] [ ]

Situation: The database CPU is saturated because a product page receives millions of repeated reads.

"What would you change first?"

Answer: I would verify that the repeated reads return data that can tolerate some staleness, then add caching at the most effective layer: CDN for public responses, application or distributed cache for shared objects, and request coalescing for simultaneous misses. I would define keys, TTL, invalidation, and behavior when the cache fails before rollout. Index tuning and read replicas may also help, but they still serve every request. I would watch cache hit rate, database load, and stale-data complaints. The design should prevent a cold-cache stampede so the cache does not merely postpone the same database overload.

S017 Write throughput has reached one database limit [ ] [ ] [ ]

Situation: A single primary database cannot sustain growing write traffic.

"What options would you evaluate before sharding?"

Answer: I would first reduce unnecessary writes, batch or debounce updates, optimize indexes, shorten transactions, and move noncritical side effects to asynchronous processing. I would confirm whether storage, CPU, locks, or connection count is the true limit. Vertical scaling and a stronger database configuration may buy time. Sharding is justified when one primary cannot meet the durable write requirement, but it introduces routing, rebalancing, cross-shard query, and transaction complexity. I would choose a stable, high-cardinality shard key, plan migrations and hot-key handling, and keep an escape path for repartitioning.

S018 Scale-down causes instability [ ] [ ] [ ]

Situation: After traffic falls, autoscaling removes instances and latency immediately oscillates up and down.

"How would you stabilize the autoscaler?"

Answer: The policy may react too quickly or to a noisy metric. I would inspect cooldown periods, stabilization windows, minimum replicas, startup and warm-up time, and whether average CPU hides overloaded instances. Scale-down should be slower than scale-up and should drain connections gracefully. I would use a demand signal that leads saturation, such as concurrency or queue depth, and require sustained low utilization before removal. I would also keep enough spare capacity for normal bursts. The goal is not minimum instance count every minute; it is predictable service with acceptable cost and without repeated cold starts.

S019 Capacity plan for a launch [ ] [ ] [ ]

Situation: Product expects a new feature to attract five million users in its first week, but the forecast is uncertain.

"How would you create a capacity plan?"

Answer: I would turn the product forecast into read and write requests per second, peak-to-average ratio, payload size, storage growth, fan-out work, and dependency calls. I would document assumptions and produce low, expected, and high scenarios instead of one precise number. Load tests should identify service and dependency limits with safety margin, while dashboards and feature flags support controlled rollout. I would pre-provision slow-scaling resources, confirm quotas, and define degradation and rollback thresholds. After launch, I would replace estimates with measured demand and revise the model rather than treating the initial spreadsheet as truth.

S020 Backpressure during overload [ ] [ ] [ ]

Situation: A service accepts work faster than it can process it, memory grows, and eventually the process crashes.

"How would backpressure change the behavior?"

Answer: Backpressure makes overload visible to callers instead of allowing an unbounded queue to consume memory. I would bound the queue and concurrency, reject or delay excess work with a clear retry signal, and prioritize critical requests. If asynchronous processing is suitable, a durable broker can absorb a limited burst, but queue depth and oldest-message age still need hard thresholds. Callers should use capped retries with jitter so rejection does not become a retry storm. Capacity must ultimately match sustained arrival rate; backpressure protects the system while traffic is reduced or processing capacity is restored.

3. Databases, Transactions, and Data Modeling

S021 Slow query after data growth [ ] [ ] [ ]

Situation: A query that took 40 ms now takes 2 seconds after the table grew from one million to one hundred million rows.

"How would you investigate and fix it?"

Answer: I would capture the exact query and execution plan with production-like parameters, because data distribution can change the chosen plan. I would inspect scanned rows, index usage, sorts, joins, filters, and whether statistics are current. The right fix may be a composite or covering index, a rewritten query, pagination, partition pruning, or precomputed data; adding every possible index would slow writes and consume storage. I would test the proposed plan on realistic volume and watch write cost after deployment. A query-latency alert by shape can reveal the next regression before users do.

S022 Duplicate records after retries [ ] [ ] [ ]

Situation: Clients retry a create request after timeouts, and the database contains duplicate orders.

"How would you make the operation safe to retry?"

Answer: I would make the create operation idempotent by requiring or generating an idempotency key that represents the client's logical request. The server stores the key with the result under a uniqueness constraint and returns the original result when the same key is retried. The check and write must be atomic; an in-memory map is not enough across instances. I would define key scope and expiry and reject reuse with different payloads. Timeouts can still leave the client uncertain, but a status lookup by key lets it learn the committed result without creating another order.

S023 Deadlocks in production [ ] [ ] [ ]

Situation: Database logs show deadlocks during peak traffic, and some requests fail even though CPU is normal.

"What causes this, and how would you reduce it?"

Answer: A deadlock occurs when transactions hold locks the others need and none can proceed. I would inspect the database deadlock graph to identify statements, tables, and lock order, then correlate them with application transactions. Immediate protection is to retry the aborted transaction a small number of times with jitter, but retries do not remove the cause. I would make code acquire resources in a consistent order, shorten transactions, reduce rows locked, add useful indexes, and move external calls outside the transaction. I would test concurrent execution because single-request tests rarely expose the cycle.

S024 Read replica returns stale data [ ] [ ] [ ]

Situation: A user updates their profile and immediately reads the old value because the read went to a replica.

"How would you provide a better user experience?"

Answer: Replication is asynchronous, so the replica can lag behind the primary. For read-after-write behavior, I could route that user's reads to the primary for a short period, attach a version and wait until a replica has applied it, or update the client optimistically when safe. I would not send every read to the primary unless the consistency requirement justifies the capacity cost. The chosen policy should be explicit per operation: account changes may need fresh reads, while feed browsing can tolerate staleness. Replication-lag metrics and fallback behavior are necessary when lag exceeds the expected window.

S025 Hot database shard [ ] [ ] [ ]

Situation: Data is sharded by user ID, but one celebrity account produces far more reads and writes than any other shard key.

"How would you handle this hot key?"

Answer: More shards do not help if all work for one key still lands on one shard. I would separate the read and write problems: aggressively cache public reads, serve media through a CDN, batch counters, and move fan-out work to queues. If the data model permits, I could split a hot logical key into subpartitions using another dimension such as post or time, while maintaining a directory for routing. That adds read aggregation and migration complexity, so I would reserve it for sustained hotspots. Rate limits and priority controls protect the primary while the data is redistributed.

S026 Cross-shard transaction requirement [ ] [ ] [ ]

Situation: A transfer must debit an account on one shard and credit an account on another shard.

"How would you preserve correctness?"

Answer: I would first ask whether related accounts can be colocated or whether the workflow can use a ledger on one authoritative partition. If not, a distributed transaction can provide atomicity but increases coordination, latency, and failure modes. A saga is often more practical: record a durable transfer intent, perform each step idempotently, and compensate or reconcile if a later step fails. The user-visible state must distinguish pending from complete rather than pretending the transfer is instantaneous. An immutable ledger, uniqueness keys, and a reconciliation job provide evidence and repair when messages or workers fail.

S027 Schema change on a large table [ ] [ ] [ ]

Situation: You need to add a required column to a table with hundreds of millions of rows without downtime.

"How would you roll out the migration safely?"

Answer: I would use an expand-and-contract sequence. First add a nullable column or a compatible default without requiring old code to write it. Deploy code that can read both forms and writes the new field, then backfill historical rows in small throttled batches while monitoring locks, replication lag, and database load. After validation, enforce the constraint and later remove old behavior. I would make the backfill restartable and pause it during pressure. A single blocking migration that rewrites the table may be operationally simple but is unsafe when its lock or duration is uncertain.

S028 Pagination slows on later pages [ ] [ ] [ ]

Situation: An API uses `OFFSET` pagination, and page 5,000 is much slower than page 1.

"What would you change?"

Answer: Large offsets force the database to find and discard many earlier rows, so cost grows with the page number and concurrent inserts can shift results. I would use keyset pagination with a stable ordering, such as `(created_at, id)`, and return an opaque cursor containing the last seen values. The next query asks for rows after that cursor using a matching index. This is fast and stable for forward navigation but does not naturally support arbitrary page numbers. I would document that trade-off and include a unique tie-breaker so equal timestamps do not duplicate or skip rows.

S029 Cache and database disagree after a write [ ] [ ] [ ]

Situation: A profile update succeeds in the database, but some users continue seeing the old cached value.

"How would you design the write and invalidation flow?"

Answer: The database should remain the source of truth, and the cache policy must define the failure window. A common approach is cache-aside: commit the database update, then invalidate the cache key so the next read reloads fresh data. Deleting is safer than writing the cache before the database because a failed transaction could publish nonexistent state. There is still a race with concurrent reads, so versioned values, short TTLs, or an event-driven invalidation can reduce risk. I would measure stale-read tolerance and include repair behavior when invalidation delivery fails.

S030 Data model for likes at large scale [ ] [ ] [ ]

Situation: A post may receive millions of likes, and the product needs both the count and whether the current viewer liked it.

"How would you model and serve this efficiently?"

Answer: I would store the user-post relationship with a uniqueness constraint so one user can like a post once, partitioned to support the dominant lookup. The exact count can be computed from relationships at small scale, but at high scale I would maintain a counter asynchronously or atomically with clear reconciliation because hot posts create write contention. Viewer state should be fetched in batches for all posts on a page, not one query per post. The displayed count may be slightly stale if product permits, while the user's own like action should reflect immediately and remain idempotent.

4. Caching and Content Delivery

S031 Cache stampede after expiry [ ] [ ] [ ]

Situation: A popular cache key expires and thousands of requests hit the database at once.

"How would you prevent this stampede?"

Answer: I would avoid letting every miss independently rebuild the same value. Options include request coalescing with one loader, a short distributed lock with timeout, stale-while-revalidate, proactive refresh, and randomized TTLs so many keys do not expire together. The fallback must be bounded; waiting forever for a failed loader simply moves the outage. I would monitor miss rate, loader latency, and database impact, and test cold-start behavior. For extremely hot content, serving a slightly stale value is often safer than forcing the origin to recompute it during a burst.

S032 Cache hit rate suddenly drops [ ] [ ] [ ]

Situation: Cache hit rate falls from 92 percent to 35 percent after a release, while traffic is unchanged.

"What would you investigate first?"

Answer: I would compare key construction, namespace or version changes, TTLs, eviction rate, memory pressure, serialization failures, and whether the release changed request cardinality. A harmless-looking new key component can turn one shared entry into millions of unique entries. I would split hit rate by endpoint and instance and check whether clients are reaching the same cache cluster. If origin load is dangerous, I would roll back or temporarily restore the old key strategy. Afterward, I would add cardinality and eviction alerts so a syntactically valid but ineffective cache is detected quickly.

S033 Cache contains sensitive data [ ] [ ] [ ]

Situation: A shared cache accidentally serves one user's private profile data to another user.

"What failed, and how would you contain it?"

Answer: The cache key or response-cache policy failed to include the authorization context. I would disable or purge the affected cache immediately, stop serving the endpoint if privacy exposure continues, and start an incident process to determine scope. The fix is to avoid shared caching for user-specific responses or include a stable user and permission dimension without putting secrets in the key. Authorization must still run on every request; a cache hit is not proof of access. I would add cross-user isolation tests, safe cache headers, audit evidence, and a review of any other personalized endpoints using the same layer.

S034 Redis becomes unavailable [ ] [ ] [ ]

Situation: The distributed cache is down, and all requests fall back to the primary database.

"How would you avoid turning a cache outage into a database outage?"

Answer: I would treat the cache as a load-bearing dependency even if data can theoretically be recomputed. The application should fail fast on cache calls, cap fallback concurrency, use local or stale values for safe data, and shed optional traffic before the database is saturated. Circuit breakers prevent every request from waiting on the failed cache, while gradual cache recovery avoids a synchronized refill. I would monitor database headroom and temporarily restrict expensive endpoints. Long term, I would test cache-loss scenarios and size the protected origin path for a deliberate, limited fallback rather than unlimited pass-through.

S035 CDN serves old content after update [ ] [ ] [ ]

Situation: A user replaces an image, but the CDN continues serving the old bytes from the same URL.

"How would you fix the design?"

Answer: Reusing a mutable URL conflicts with long-lived edge caching. I would store media under an immutable, content- or version-based URL and update the database reference when a new image is ready. Old URLs can remain cached safely and expire later. If immediate correction is required, a CDN purge can contain the issue, but purges are slower, costly, and harder to reason about at scale. I would separate metadata from media bytes so profile data can change quickly while immutable objects receive long cache lifetimes. Clients should not guess versions; the API returns the current canonical URL.

S036 Choosing a cache TTL [ ] [ ] [ ]

Situation: The team asks for one standard TTL for every cached object.

"How would you choose TTLs instead?"

Answer: TTL should follow each object's freshness requirement, update frequency, recomputation cost, popularity, and impact of stale data. Public profile images can live much longer than account permissions or inventory. I would define a bounded staleness target with product and security stakeholders, then use invalidation when prompt changes matter and TTL as a safety net. I would add jitter to reduce synchronized expiry and measure both hit rate and stale-read incidents. One global TTL is easy to configure but either wastes cache value on stable data or keeps volatile data stale for too long.

S037 Hot key overloads one cache node [ ] [ ] [ ]

Situation: A single viral post key receives most requests and saturates one cache node.

"How would you distribute the load?"

Answer: Standard key partitioning cannot spread one key because all requests hash to the same owner. I would first serve eligible content through a CDN or application-local cache so requests do not reach the distributed cache. Request coalescing and stale values reduce refill pressure. For very hot small objects, controlled key replication or sharded replicas can spread reads, but updates and invalidation become more complex. I would detect hot keys directly rather than relying only on average node load. The design should also protect the database if the hot entry disappears or the cache node fails.

S038 Cache warming during deployment [ ] [ ] [ ]

Situation: Every new instance starts with an empty local cache, causing a latency spike during deployments.

"How would you make startup safer?"

Answer: I would keep new instances out of full traffic until readiness checks pass and a limited warm-up completes. Warm only the small set of predictable hot data; loading the entire cache delays startup and may overload dependencies. A shared cache or stale snapshot can reduce per-instance misses, and a slow traffic ramp lets real requests populate entries gradually. I would randomize rollout timing and monitor origin load. The instance should still work correctly on a miss, because prewarming is an optimization rather than a correctness requirement, and deployment rollback must not depend on completing it.

S039 Negative caching for missing data [ ] [ ] [ ]

Situation: Bots repeatedly request nonexistent usernames, causing expensive database misses.

"Would you cache not-found results?"

Answer: Yes, if I use a short negative-cache TTL and understand the creation race. Caching a not-found result prevents repeated database lookups for the same invalid key, but a long TTL could hide a username created moments later. I would invalidate the negative entry when creation succeeds or keep the TTL short enough for the product requirement. I would distinguish a real not-found response from a database error so failures are not cached as absence. Rate limiting remains necessary because attackers can generate unlimited unique names and bypass per-key caching.

S040 Local cache versus distributed cache [ ] [ ] [ ]

Situation: The team wants the lowest possible latency and proposes replacing Redis with per-instance memory caches.

"What trade-offs would you explain?"

Answer: Local caches are extremely fast and remove a network hop, but each instance holds different data, duplicates memory, and needs a strategy for invalidation and cold starts. A distributed cache provides shared state and better aggregate memory use but adds network latency and another service dependency. I would choose based on consistency and access patterns, and often combine them: a very small short-lived local cache in front of a distributed cache. I would prevent unbounded memory growth and measure stale behavior. If correctness depends on globally current values, neither cache should replace the authoritative datastore.

5. Messaging, Queues, and Asynchronous Work

S041 Queue backlog grows continuously [ ] [ ] [ ]

Situation: Producers publish 5,000 events per second, consumers process 3,000, and queue lag keeps growing.

"What would you do immediately and long term?"

Answer: The system is unstable because sustained arrival rate exceeds service rate. I would measure oldest-message age, partition lag, consumer errors, processing time, and downstream saturation before simply adding consumers. Immediate actions may include scaling consumers, pausing noncritical producers, prioritizing urgent events, and shedding optional work. If one partition or dependency is the limit, more consumers will not help. Long term, I would optimize or batch processing, choose enough partitions, enforce producer quotas, and define maximum acceptable lag. A queue absorbs bursts; it cannot solve a permanent throughput deficit.

S042 Duplicate message delivery [ ] [ ] [ ]

Situation: A consumer occasionally processes the same payment event twice after a restart.

"How would you make the consumer safe?"

Answer: Most practical brokers provide at-least-once delivery, so duplicates are expected when acknowledgement and processing are not one atomic action. I would make the business operation idempotent using a stable event or command ID and a durable uniqueness record in the same transaction as the state change. The consumer should acknowledge only after the transaction commits. For external side effects, I would use an outbox or provider idempotency key. I would test redelivery deliberately. Broker settings may reduce duplicates, but correctness should not depend on a promise that a message will arrive exactly once.

S043 Poison message blocks a partition [ ] [ ] [ ]

Situation: One malformed event fails repeatedly and prevents later events in the same partition from being processed.

"How would you recover without silently losing data?"

Answer: I would limit retries with exponential backoff, capture the failure reason, and move the event to a dead-letter queue after the threshold so the partition can continue. The dead-letter record must preserve payload, headers, offset, schema version, and error context, with alerts and an owner. I would fix the consumer or data, then replay through a controlled idempotent path rather than editing offsets blindly. For ordering-sensitive workflows, skipping may violate correctness, so I might quarantine that key instead of the entire partition. The recovery policy should be designed before the first poison message.

S044 Event published but database write fails [ ] [ ] [ ]

Situation: A service publishes `OrderCreated`, then its database transaction fails, leaving consumers with an event for an order that does not exist.

"How would you prevent this inconsistency?"

Answer: I would avoid a dual write that requires the database and broker to succeed independently. With a transactional outbox, the service writes the order and an outbox row in one database transaction. A separate publisher reads committed outbox rows and sends them to the broker, retrying safely with event IDs. Consumers remain idempotent because publication can repeat. The reverse problem is handled too: an order cannot commit without an outbox record. This adds a relay, cleanup, and monitoring requirement, but it provides a durable bridge without a distributed transaction between unrelated systems.

S045 Consumer is fast but downstream is failing [ ] [ ] [ ]

Situation: Queue consumers rapidly retry calls to a failing email provider, increasing provider errors and queue lag.

"How would you stop the retry storm?"

Answer: I would add timeouts, capped retries with exponential backoff and jitter, and a circuit breaker that stops calls while the provider is unhealthy. Failed messages can be delayed or moved to a retry topic so they do not block fresh work or occupy workers. I would respect provider rate limits and separate permanent errors, such as invalid addresses, from temporary failures. During the outage, the queue provides durability, but I would watch retention and oldest-message age. Recovery should ramp gradually so accumulated messages do not immediately overload the provider again.

S046 Message ordering requirement [ ] [ ] [ ]

Situation: Profile update events sometimes arrive out of order, and an older event overwrites newer data.

"How would you preserve the correct final state?"

Answer: I would partition events by the entity key so updates for one profile are normally ordered, but I would still include a monotonically increasing version or source timestamp because retries and replays can reorder delivery. The consumer applies an event only if its version is newer than the stored version. Global ordering is unnecessary and expensive when only per-profile order matters. I would define how concurrent writers obtain versions and how deletes are represented. Monitoring rejected stale events helps detect upstream problems without allowing old state to overwrite current data.

S047 Exactly-once request from product [ ] [ ] [ ]

Situation: Product says a notification must be delivered "exactly once."

"How would you clarify and design for that requirement?"

Answer: I would ask whether the requirement means no duplicate user-visible notification, no duplicate provider call, or no duplicate internal event. End-to-end exactly-once delivery is rarely guaranteed across networks and external providers because acknowledgements can be lost. I would design at-least-once processing with stable notification IDs, idempotent state changes, deduplication at send time, and a durable delivery record. Some rare ambiguity may remain at the external boundary. I would explain that we can make duplicates extremely unlikely and observable, but claiming absolute exactly-once behavior without a shared transaction would be misleading.

S048 Replaying historical events [ ] [ ] [ ]

Situation: A new search indexer must rebuild its state from six months of events without affecting live traffic.

"How would you run the replay safely?"

Answer: I would use a separate consumer group and isolated capacity, starting from the required offset or archived event store. The indexer must be idempotent and schema-aware because old events may use earlier formats. I would throttle replay based on broker, database, and search-cluster headroom, and keep live updates flowing so the rebuilt index can catch up. Counts, checksums, and sampled comparisons validate completeness before traffic switches. If the source log does not retain six months, I would seed from a consistent database snapshot and replay events after the snapshot boundary.

S049 Choosing a queue partition key [ ] [ ] [ ]

Situation: You need to partition a high-volume event topic for user activity.

"How would you choose the partition key?"

Answer: The key should preserve only the ordering the business needs while distributing load evenly. User ID is suitable when events for one user must stay ordered and user activity is reasonably balanced. A constant key preserves global order but destroys parallelism; a random key scales well but loses per-user order. I would estimate key cardinality and skew, especially celebrity or bot accounts, and choose enough partitions for future throughput because repartitioning is operational work. Consumers and downstream stores must use the same ownership assumptions so state is not concurrently updated without coordination.

S050 Synchronous endpoint does too much work [ ] [ ] [ ]

Situation: Creating a post waits for image processing, search indexing, analytics, and notifications before returning.

"How would you redesign the request flow?"

Answer: I would keep only the work required to durably accept the post in the synchronous path: validate authorization, store metadata, and record reliable events or jobs. The API can return an accepted or processing state while workers generate media variants, update search, record analytics, and send notifications. Each worker must be idempotent, observable, and able to retry independently. The UI should show progress and a clear failure state instead of pretending all side effects completed. This lowers user latency and isolates failures, but it introduces eventual consistency and requires durable workflow tracking.

6. Reliability, Resilience, and Disaster Recovery

S051 Downstream dependency is slow [ ] [ ] [ ]

Situation: A recommendation service becomes slow and starts consuming all request threads in the feed API.

"How would you prevent the dependency from taking down the caller?"

Answer: I would set a timeout based on the caller's latency budget, limit concurrent calls with a bulkhead, and open a circuit breaker when failures or latency cross a threshold. The feed should have a defined fallback, such as cached recommendations, chronological content, or omitting the section. Retries must be few, budget-aware, and jittered because retrying a saturated dependency amplifies load. I would monitor fallback rate as a degraded-service signal. The long-term fix may be dependency scaling or a looser asynchronous design, but the caller must remain protected whenever the dependency fails again.

S052 Retry storm after a brief outage [ ] [ ] [ ]

Situation: A database is unavailable for 30 seconds, then receives a huge burst of retries and remains overloaded after recovery.

"What retry policy would avoid this?"

Answer: I would use exponential backoff with random jitter, a strict maximum attempt count, and a total request deadline. Retries should occur only for operations known to be transient and safe to repeat; writes need idempotency. A circuit breaker can stop futile calls during the outage, while concurrency limits protect the recovering database. Clients should not all retry at the same fixed interval. I would also review timeouts: a long timeout plus several retries can exceed user patience and hold resources. Recovery should reopen gradually so the database proves it has headroom before receiving full traffic.

S053 One dependency fails but core service should remain usable [ ] [ ] [ ]

Situation: The analytics service is down, but users should still be able to upload photos.

"How would you degrade gracefully?"

Answer: Analytics should not be on the critical upload path. I would durably accept the upload and publish or store an analytics event asynchronously. If the broker is unavailable, an outbox in the upload database can preserve the event without blocking the user indefinitely. The application should apply a short timeout and skip nonessential synchronous analytics calls. I would expose a degraded-state metric and alert because silent loss is not acceptable. Once analytics recovers, it can replay idempotently. The design explicitly separates product correctness from secondary measurement so an optional dependency cannot become mandatory by accident.

S054 Multi-region failover decision [ ] [ ] [ ]

Situation: The primary region is experiencing widespread network failures, and traffic may need to move to a secondary region.

"What would you verify before failing over?"

Answer: I would confirm the failure scope, secondary-region health and capacity, data replication lag, DNS or routing controls, and whether both regions could accept conflicting writes. If the system is active-passive, I would fence the old primary or make it read-only before promoting the secondary to avoid split brain. I would compare expected data loss with the recovery point objective and expected downtime with the recovery time objective. Traffic should shift gradually where possible. After recovery, I would reconcile data and rehearse failback rather than immediately reversing direction under pressure.

S055 Backup exists but restore was never tested [ ] [ ] [ ]

Situation: The team takes daily backups and assumes disaster recovery is covered.

"Why is that insufficient?"

Answer: A backup is useful only if it is complete, protected, and restorable within the required time. I would schedule automated restore tests into an isolated environment, verify checksums and schema compatibility, and run application-level checks on restored data. I would measure actual restore duration and data age against recovery objectives. Credentials, encryption keys, configuration, and dependent object storage may also be required; a database file alone may not recreate the service. I would retain multiple versions in a separate failure domain and alert on missed backups. Recovery evidence matters more than a successful backup job status.

S056 Partial network partition [ ] [ ] [ ]

Situation: Half the service instances can reach the database and half cannot, while all still pass basic process health checks.

"How should health checks and routing behave?"

Answer: A liveness check should confirm the process can make progress without causing constant restarts, while readiness should remove an instance from traffic when it cannot serve required requests. I would make readiness reflect critical dependency reachability with careful timeouts and hysteresis so brief failures do not flap every instance at once. If some endpoints can work without the database, separate routing or degraded behavior may preserve them. I would compare network paths and zones to isolate the partition. The system should avoid reporting success while requests fail, but health checks must not create a synchronized self-evacuation during dependency pressure.

S057 Leader election split brain [ ] [ ] [ ]

Situation: Two scheduler instances both believe they are leader and execute the same cleanup job.

"How would you prevent duplicate leadership from corrupting data?"

Answer: Leader election alone is not enough because an old leader can continue working after losing its lease during a pause or partition. I would use a lease with expiry and a monotonically increasing fencing token. Every protected write includes the token, and the storage layer rejects work from an older leader. Jobs should also be idempotent when possible. I would inspect clock assumptions and use a proven coordination service rather than custom in-memory voting. Monitoring should alert on overlapping leadership and duplicate job IDs so the protection is tested before a destructive task exposes the flaw.

S058 Cascading failure across services [ ] [ ] [ ]

Situation: One slow service causes callers to retry, their thread pools fill, and unrelated endpoints begin failing.

"How would you stop the cascade?"

Answer: I would contain the initial dependency with short bounded timeouts, circuit breakers, bulkheads, and concurrency limits so its callers retain resources for other work. Retries should respect a shared deadline and retry budget rather than multiplying at every layer. Load shedding and priority queues keep critical operations available. I would trace the dependency graph to identify where latency and retries amplify, then disable optional calls or use cached fallbacks. Long term, I would run failure tests that slow one dependency and verify the blast radius, because ordinary happy-path load tests do not expose cascading behavior.

S059 Disaster destroys primary data store [ ] [ ] [ ]

Situation: The primary database and its local replicas are permanently lost.

"Walk through recovery priorities."

Answer: I would declare the incident, stop unsafe writes, and confirm the latest restorable backup or cross-region replica and its data age. Recovery follows the documented runbook: provision a clean store, restore data, apply logs if available, validate integrity and application compatibility, then direct limited traffic while watching errors. I would communicate the expected recovery point and any confirmed data loss rather than guessing. After service restoration, reconciliation may recover operations from durable queues or external records. The post-incident work must address why replicas and backups shared a failure domain and repeat the restore drill.

S060 Graceful shutdown during deployment [ ] [ ] [ ]

Situation: Instances terminate during rolling deployment and active requests fail with connection resets.

"How would you make termination graceful?"

Answer: On termination, the instance should first fail readiness so the load balancer stops sending new traffic, then wait for routing changes to propagate and drain active requests within a bounded grace period. Background consumers should stop taking new messages, finish or safely abandon current work, and acknowledge only committed work. Long-lived connections may need explicit close signals or reconnection guidance. I would align application shutdown timeouts with platform termination settings and test slow requests. Graceful shutdown reduces expected errors, but hard deadlines remain necessary so a stuck process cannot block deployment indefinitely.

7. APIs, Consistency, and Correctness

S061 Client retries a timed-out write [ ] [ ] [ ]

Situation: A client times out while creating a post and does not know whether the server committed it.

"What should the API allow the client to do?"

Answer: The API should accept an idempotency key so the client can safely retry the same logical create. The server atomically stores the key, request identity, status, and result with the post. If the original request committed, the retry returns the same post; if it did not, one attempt creates it. A status endpoint by key can resolve longer-running uncertainty. I would reject the same key with a different payload and define retention. Without this contract, the client must choose between duplicate posts and potentially losing a valid user action.

S062 API version breaks old mobile clients [ ] [ ] [ ]

Situation: A backend response removes a field, and users on older mobile versions start crashing.

"How would you manage compatibility?"

Answer: Mobile clients can remain in the field for months, so server changes should be backward compatible by default. I would restore the field or roll back immediately, then use additive changes, tolerant readers, defaults, and explicit deprecation metrics. If a breaking contract is unavoidable, I would introduce a versioned endpoint or negotiated representation and maintain both until usage falls below an agreed threshold. Contract tests using older client expectations should run before deployment. I would avoid reusing a field with new meaning because syntactic compatibility can still create incorrect behavior.

S063 Concurrent updates overwrite each other [ ] [ ] [ ]

Situation: Two devices edit the same profile, and the later database write silently erases the earlier change.

"How would you prevent lost updates?"

Answer: I would use optimistic concurrency with a version number or ETag. The client reads version 12 and submits an update conditional on version 12; the database updates only if the current version still matches and increments it. If another write already produced version 13, the server returns a conflict so the client can reload, merge, or ask the user. This avoids long-held locks for normal low-contention edits. Field-level patch semantics may reduce conflicts, but the API must define whether absent fields mean unchanged or cleared. Silent last-write-wins is acceptable only when the product explicitly accepts data loss.

S064 Strong versus eventual consistency for follower count [ ] [ ] [ ]

Situation: A user follows another account, but the displayed follower count updates a few seconds later.

"Is eventual consistency acceptable here?"

Answer: I would separate relationship correctness from aggregate display. The follow edge should be committed durably and idempotently so authorization and future actions see the correct relationship. The public counter can often update asynchronously and be temporarily stale because exact immediate counts are not usually critical. The initiating user's UI can update optimistically while reconciliation corrects drift. I would document the expected delay and monitor lag. Strong consistency is worth its coordination cost for money, permissions, or uniqueness, but applying it to every social counter would reduce availability and throughput without proportional user value.

S065 Partial success in a batch API [ ] [ ] [ ]

Situation: A batch request contains 100 items; 93 succeed and 7 fail validation.

"What should the API return and retry behavior be?"

Answer: I would define atomicity explicitly. If items are independent, the response should report per-item status, stable identifiers, and actionable errors so the client retries only failed transient items. Successful items must not repeat, so idempotency keys or item IDs are required. If business rules require all-or-nothing behavior, the server should validate before commit or use a transaction within a supported boundary, accepting the capacity cost. A vague 500 response encourages retrying all 100 items and creating duplicates. Metrics should distinguish validation failures from infrastructure failures because their remedies differ.

S066 API returns too much data [ ] [ ] [ ]

Situation: A feed endpoint returns full comments, likes, profiles, and media metadata for every post, creating multi-megabyte responses.

"How would you redesign the contract?"

Answer: I would identify what the initial screen actually needs and return a bounded summary: post fields, a small comment preview, viewer state, and counts. Additional comments or details load through paginated endpoints. I would support field selection only if the complexity is justified and ensure authorization remains consistent. Compression helps bandwidth but does not remove database, serialization, memory, or client parsing cost. I would set response-size and item-count limits, measure payload percentiles, and version the contract compatibly. The goal is predictable work per request rather than an unbounded nested object graph.

S067 Webhook receiver sees duplicate callbacks [ ] [ ] [ ]

Situation: A payment provider sends the same webhook several times and sometimes out of order.

"How would you process it safely?"

Answer: I would verify the provider signature and timestamp before accepting the event, then store the provider event ID under a uniqueness constraint. Processing is idempotent, and state transitions validate version or allowed order so an older event cannot reverse newer state. I would acknowledge quickly after durable acceptance and perform slow work asynchronously, because a slow response triggers more retries. Unknown events should be retained for investigation rather than treated as success without evidence. A reconciliation job against the provider API closes gaps when callbacks are delayed or permanently missed.

S068 Public API rate limiting [ ] [ ] [ ]

Situation: A public endpoint is overwhelmed by one client sending thousands of requests per second.

"How would you design rate limiting?"

Answer: I would choose the identity and scope first: API key, user, IP, tenant, endpoint, or a combination. A token-bucket limit allows bounded bursts while controlling the sustained rate. Limits should be stricter for expensive writes than cheap reads and should return a clear status with retry guidance. A distributed limiter needs shared or partition-consistent state, while local limits are faster but approximate globally. I would include abuse detection, allowlists for trusted internal traffic, and metrics for rejected requests. The limiter protects capacity but does not replace authentication or endpoint optimization.

S069 Search endpoint supports arbitrary filters [ ] [ ] [ ]

Situation: Product wants users to combine dozens of filters and sorts on a rapidly growing dataset.

"Would you keep querying the primary relational database?"

Answer: I would start with actual query patterns, scale, freshness, and correctness needs. A relational database with targeted indexes may be simplest for modest data and transactional filters, but arbitrary full-text search and many combinations can create expensive plans and index explosion. A search engine offers inverted indexes and ranking but introduces asynchronous indexing, operational cost, and temporary inconsistency. I would keep the database authoritative, publish changes reliably, and rebuild the search index from a snapshot plus events. The API should expose bounded supported filters rather than promising every possible combination.

S070 Delete must propagate across systems [ ] [ ] [ ]

Situation: A user deletes a post that exists in the database, cache, search index, CDN, analytics pipeline, and backups.

"How would you design deletion?"

Answer: I would make the database deletion or tombstone the authoritative state and immediately prevent the post from being returned or authorized. A durable deletion event drives idempotent removal from caches, search, derived stores, and media references, with retries and completion tracking. CDN content may require purge or short-lived access controls; backups follow a documented retention and erasure policy rather than instant mutation. I would distinguish user-visible deletion latency from eventual physical cleanup and provide audit evidence. A reconciliation job finds missed consumers so one failed event does not leave content discoverable indefinitely.

8. Deployments, Observability, and Operations

S071 Canary looks healthy but full rollout fails [ ] [ ] [ ]

Situation: A 5 percent canary shows no problems, but latency rises after the release reaches 70 percent.

"Why did the canary miss it, and how would you improve the rollout?"

Answer: The regression may depend on aggregate load, cache churn, connection count, a rare traffic segment, or interaction between new instances. I would pause or roll back, then compare version-specific metrics and dependency pressure at each rollout step. Future canaries should receive representative traffic, run long enough to cover background behavior, and include guardrails for downstream metrics, not only canary CPU and errors. I would use progressive stages such as 1, 5, 20, 50, and 100 percent with automatic analysis and soak time. Some capacity failures require load testing in addition to canaries.

S072 Metrics are green but users report failures [ ] [ ] [ ]

Situation: Standard dashboards show normal CPU, average latency, and error rate, yet customer support reports many failed uploads.

"What might observability be missing?"

Answer: Aggregate metrics may hide one endpoint, tenant, region, file size, or client version. I would reproduce a failed request and segment success rate and latency by those dimensions, then inspect traces and structured logs using a request ID. The service may also return HTTP 200 while the asynchronous upload later fails, so business outcome metrics matter. I would add an end-to-end synthetic upload and track accepted, processed, and visible states. Observability should represent the user's journey, not only machine health, and should avoid unbounded labels such as raw user IDs.

S073 Logs are too noisy during incidents [ ] [ ] [ ]

Situation: An incident produces millions of repetitive error logs, making useful evidence difficult to find and increasing logging cost.

"How would you improve logging?"

Answer: I would use structured logs with stable fields such as service, operation, request ID, error class, dependency, and version. Expected transient failures should be sampled or aggregated, while security and data-loss events remain complete. I would avoid logging the same exception at every layer and ensure sensitive values are redacted. Metrics should count high-volume events; logs provide detailed examples. During the incident, I might temporarily reduce a noisy category without disabling audit evidence. Retention and indexing should reflect diagnostic value, and alerts should point to a focused query rather than a raw log stream.

S074 Missing distributed trace spans [ ] [ ] [ ]

Situation: A request crosses five services, but traces often stop after the second service.

"How would you diagnose the broken trace?"

Answer: I would check whether trace context is propagated through HTTP headers, messaging metadata, thread or reactive boundaries, and retry clients. Sampling decisions should remain consistent across services; otherwise children may be dropped even when the root is kept. I would verify instrumentation versions, exporters, collector capacity, and clock synchronization. A correlation ID in logs provides a fallback but is not a substitute for parent-child timing. I would add an automated end-to-end trace test across synchronous and asynchronous paths and monitor span-drop rate so observability failures are detected before a production incident.

S075 Alert fires too often and gets ignored [ ] [ ] [ ]

Situation: A latency alert pages the team several times a week but rarely requires action.

"How would you make the alert useful?"

Answer: I would tie paging to user-impacting symptoms and a documented action. Instead of a fixed average-latency threshold, I might alert on sustained p95 or p99 latency combined with request volume or error-budget burn. Short spikes can create tickets or dashboards rather than waking someone. I would review historical pages, remove redundant alerts, and include service, scope, likely dependencies, and runbook links. After every page, the owner should classify whether it was actionable. An alert that nobody trusts is not harmless; it delays response to the incident that actually matters.

S076 Need to roll back a database-compatible release [ ] [ ] [ ]

Situation: Application code must roll back, but the deployment already started writing a new database field.

"How do you make rollback safe?"

Answer: The migration and code should have been designed for mixed versions. During expansion, old code must tolerate the new field and new code should continue writing any old representation needed by the previous version. If that contract holds, I can roll application instances back without reversing the schema. Destructive cleanup happens only after the rollback window closes and all readers have migrated. If compatibility is uncertain during an incident, I would stop the rollout and test the previous version against current data before broad rollback. Schema rollback is usually riskier than leaving an unused additive column temporarily.

S077 Feature flag causes inconsistent behavior [ ] [ ] [ ]

Situation: Two services evaluate the same feature flag differently, producing requests one service cannot understand.

"How would you manage cross-service feature rollout?"

Answer: A flag is not a substitute for a compatible protocol. I would make the receiving service understand both old and new requests before any caller sends the new form. Flag ownership, targeting, defaults, and expiry should be explicit, and evaluation should use a consistent subject key when user experience must be stable. During the incident, I would disable the producer-side behavior or route only to compatible versions. I would audit configuration propagation delay and fail-safe defaults. Once rollout completes, I would remove the flag and old path so temporary branching does not become permanent architecture.

S078 Configuration change causes outage [ ] [ ] [ ]

Situation: No code was deployed, but a connection-pool configuration change caused widespread timeouts.

"How should configuration changes be managed?"

Answer: Configuration can change behavior as significantly as code, so it needs version control, review, validation, staged rollout, observability, and rollback. I would restore the previous value first if impact is clear, then verify whether the new pool size exhausted the database or created local waiting. Sensitive and environment-specific values remain in a secure configuration system, but their schemas and safe ranges can still be tested. I would record who changed what and when, compare effective configuration by instance, and use canary application of high-risk settings rather than broadcasting them globally.

S079 Load test passes but production fails [ ] [ ] [ ]

Situation: A service passes a synthetic load test at expected requests per second but fails under real traffic.

"What might the test have missed?"

Answer: Request rate alone is not a representative workload. I would compare payload sizes, endpoint mix, key distribution, cache warmth, connection reuse, think time, authenticated behavior, database volume, and dependency latency. Production may contain hot keys, large media, rare slow queries, or synchronized bursts that the test did not model. I would use sanitized traffic statistics to build multiple workload profiles and run long enough to expose memory or pool leaks. The result should report saturation points and failure behavior, not only a pass at one target. Production can then validate assumptions through a controlled canary.

S080 Runbook is outdated during an incident [ ] [ ] [ ]

Situation: The on-call engineer follows a runbook, but commands and dashboards no longer match the system.

"How would you keep operational documentation trustworthy?"

Answer: I would stop using unsafe steps, stabilize the service with current expertise, and record exactly where the runbook failed. Runbooks need named owners, review dates, tested commands, expected observations, rollback steps, and links that are validated automatically where possible. Incident drills and actual incidents should update them immediately. I would prefer small task-focused procedures over a large document nobody can rehearse. Automation should encode repetitive safe actions, but the runbook must still explain intent and decision boundaries. A stale runbook should be marked unverified rather than presented as authoritative.

9. Security, Privacy, and Abuse

S081 Credential-stuffing attack [ ] [ ] [ ]

Situation: Login failures rise sharply from many IP addresses using known email and password pairs.

"How would you protect accounts without locking out legitimate users?"

Answer: I would detect attempts by account, IP, device, network range, and reputation rather than relying on one global threshold. Progressive delays, risk-based challenges, breached-password checks, and multi-factor authentication can slow attackers while preserving legitimate recovery. Per-account limits should not let an attacker permanently deny service to a victim. I would avoid revealing whether an email exists and alert users on suspicious successful logins. Immediate containment may include blocking high-confidence sources and tightening limits. Long term, I would monitor takeover outcomes, not only failed attempts, and protect reset flows equally.

S082 Authorization bug exposes another user's data [ ] [ ] [ ]

Situation: A user changes an object ID in the URL and can view another user's private resource.

"What should happen immediately and how do you prevent recurrence?"

Answer: I would disable or restrict the affected endpoint, preserve audit evidence, determine exposure scope, and follow the security incident process. The root failure is object-level authorization: authentication proved who the caller was but the service did not verify access to that specific resource. I would centralize authorization checks close to data access, use owner or policy constraints in the query, and return the same safe response for unauthorized objects where appropriate. Tests must cover cross-user and cross-tenant access, including cached paths. Logging should identify access decisions without recording sensitive payloads.

S083 Secret appears in logs [ ] [ ] [ ]

Situation: An access token is accidentally written to centralized application logs.

"How would you contain and remediate the incident?"

Answer: I would revoke or rotate the exposed credential immediately, restrict access to the affected logs, and identify how widely the value was replicated or exported. Deleting one local file is insufficient when logs have indexes, backups, and downstream sinks. I would search for similar patterns without spreading the secret further, notify security owners, and preserve an audit trail. The code fix should use structured redaction at a common logging boundary and prohibit sensitive headers and fields. Tests and scanners can catch known secret formats, but developers also need a data-classification rule for values that scanners cannot recognize.

S084 File upload contains malicious content [ ] [ ] [ ]

Situation: Users can upload images, and an attacker uploads a file with a misleading extension and embedded malicious payload.

"How would you secure the upload pipeline?"

Answer: I would not trust the filename, extension, or client-provided content type. The service should enforce size limits, inspect file signatures, decode and re-encode supported media, scan in an isolated environment, and store uploads outside executable application paths. Newly uploaded objects remain quarantined until processing succeeds. Delivery uses a separate media domain with safe content headers and no authentication cookies. I would generate random storage keys and strip unnecessary metadata. Timeouts and resource limits protect image processors from decompression bombs, while failed or suspicious files are retained only according to a controlled investigation policy.

S085 Internal service trusts any network caller [ ] [ ] [ ]

Situation: A service assumes requests are safe because they come from the private network.

"Why is that risky, and what would you add?"

Answer: Network location is a weak identity boundary; compromised workloads, routing mistakes, or exposed proxies can make unauthorized calls. I would authenticate service identities using short-lived credentials or mutual TLS, authorize specific operations by least privilege, and encrypt traffic where required. Network policies reduce reachability but do not replace application authorization. I would rotate identities automatically and audit denied and sensitive actions. During migration, I would inventory callers and introduce enforcement gradually to avoid breaking unknown dependencies. The goal is explicit trust between workloads, not a blanket assumption that everything inside the network is safe.

S086 Multi-tenant data leakage risk [ ] [ ] [ ]

Situation: A SaaS database stores many tenants in shared tables, and the team worries about queries missing the tenant filter.

"How would you reduce the chance and impact of leakage?"

Answer: I would make tenant identity part of every authenticated request context and enforce it in a shared data-access layer rather than relying on each developer to remember a filter. Composite keys and uniqueness constraints should include tenant ID, and database row-level security can add defense in depth where supported. Cache keys, queues, object storage paths, and logs also need tenant boundaries. Automated tests should attempt cross-tenant access for every resource type. For high-risk or regulated tenants, stronger isolation such as separate schemas or databases may be justified, with added operational cost.

S087 DDoS traffic reaches application servers [ ] [ ] [ ]

Situation: A volumetric attack overwhelms network and application capacity before normal rate limits can respond.

"Where should defenses be placed?"

Answer: Volumetric traffic should be absorbed as far from the application as possible using the CDN, managed DDoS protection, edge filtering, and load-balancer limits. Application rate limiting is still useful for authenticated abuse and expensive operations, but it cannot help if network links or proxies are already saturated. I would cache public content, challenge suspicious clients, protect origin addresses, and preserve a small path for operators. The incident plan should include provider contacts and traffic baselines. Afterward, I would review which layers exhausted first and test that emergency rules do not block all legitimate users.

S088 User requests data deletion [ ] [ ] [ ]

Situation: A user asks for all personal data to be deleted from the platform.

"How would you make the deletion complete and auditable?"

Answer: I would maintain a data inventory mapping the user's identifiers across authoritative databases, derived stores, search, caches, object storage, analytics, and third parties. A durable workflow records the request, verifies authorization, deletes or irreversibly anonymizes each system idempotently, and tracks completion with retries. Backups follow the documented retention policy and prevent restored data from becoming active again. Legal retention exceptions must be explicit and access-controlled. The user-facing status should not claim completion until required systems acknowledge it, and audits should prove the workflow without retaining the deleted personal data itself.

S089 Signed media URL is shared [ ] [ ] [ ]

Situation: A private photo URL is signed for one user but can be copied and used by anyone until it expires.

"How would you choose a safer access model?"

Answer: A signed URL is a bearer credential, so anyone holding it can use it during its lifetime. I would keep expiry short, scope it to the exact object and operation, use HTTPS, and avoid placing it in logs or analytics. If stronger user binding is required, requests can pass through an authenticated media proxy or use edge authorization that validates a session, accepting extra latency and cost. I would revoke access by changing the object version or signing key only when necessary because broad key rotation affects many URLs. The design depends on privacy sensitivity and acceptable sharing risk.

S090 Audit log can be modified by application admins [ ] [ ] [ ]

Situation: The same administrators who operate the service can alter records intended to prove sensitive actions.

"How would you make audit evidence more trustworthy?"

Answer: I would send security-relevant events to an append-oriented system with separate access control, limited deletion rights, retention enforcement, and monitoring for gaps. Events include actor identity, action, target, timestamp, request or trace ID, result, and relevant policy version without unnecessary secrets. Application administrators may read operational details but should not be able to rewrite the independent record. Cryptographic chaining or immutable storage can strengthen tamper evidence for high-risk systems. I would test clock consistency, delivery failure behavior, and reconciliation so the absence of an event is detected rather than silently accepted.

10. Feeds, Media, Search, Ranking, and Notifications

S091 Celebrity fan-out overload [ ] [ ] [ ]

Situation: A celebrity with 50 million followers publishes a post, and fan-out-on-write would enqueue 50 million feed updates.

"How would you design around this hotspot?"

Answer: I would use a hybrid feed strategy. Normal accounts can fan out on write for fast reads, while celebrity posts remain in an author stream and are merged into follower feeds at read time or through staged batch distribution. This avoids one write creating an immediate massive burst, though reads become more complex and may need caching. I would define the celebrity threshold dynamically from fan-out cost, monitor queue lag, and make duplicate insertion idempotent. The ranking layer should treat pulled and pushed candidates consistently so the optimization does not change product semantics unexpectedly.

S092 Feed shows duplicate posts [ ] [ ] [ ]

Situation: Users sometimes see the same post twice after refreshing or loading the next page.

"Where could duplicates enter, and how would you remove them?"

Answer: Duplicates can come from repeated fan-out events, overlapping pagination windows, merging several candidate sources, or retries after uncertain writes. I would assign a stable post ID and deduplicate at durable insertion where possible, then deduplicate again during feed assembly as defense in depth. Cursor pagination should use a stable score plus unique tie-breaker so page boundaries do not overlap. I would trace duplicate pairs to their source rather than hiding every error in the UI. Consumer idempotency and unique feed-entry constraints prevent the stored timeline from growing with duplicate work.

S093 Feed freshness versus ranking quality [ ] [ ] [ ]

Situation: A sophisticated ranking model improves engagement but takes several seconds to update after a new post.

"How would you balance freshness and ranking?"

Answer: I would define a latency budget and combine candidate paths. A fast path can inject recent eligible posts using simple rules, while the main ranked feed uses richer features and precomputation. The merger removes duplicates and enforces safety and visibility rules. I would measure freshness, engagement, latency, and compute cost rather than optimizing one metric alone. If the model or feature service is slow, a cached score or chronological fallback keeps the feed available. The product may accept slightly lower ranking precision for the first seconds after publication to avoid making new content invisible.

S094 Image processing backlog [ ] [ ] [ ]

Situation: Original images upload successfully, but thumbnail generation is hours behind.

"How would you restore service and prevent recurrence?"

Answer: I would inspect queue age, worker errors, processing time by image type, and CPU or memory saturation. Immediate actions include scaling healthy workers, quarantining poison files, prioritizing user-visible variants, and limiting new work if storage or queues approach limits. Upload responses should show a processing state and never reference a thumbnail that does not exist. Long term, I would make jobs idempotent, enforce input limits, generate common variants first, and autoscale on oldest-job age rather than only CPU. Reprocessing tools and source-image retention provide recovery when a processor bug creates bad outputs.

S095 Video upload interrupted halfway [ ] [ ] [ ]

Situation: A user uploads a large video over an unreliable mobile connection and loses connectivity at 70 percent.

"How would you support reliable resume?"

Answer: I would use multipart or chunked upload with an upload session ID. Each part has a sequence number and checksum and can be retried idempotently, ideally directly to object storage using short-lived signed permissions. The server tracks received parts and finalizes only when all expected parts are present and validated. Incomplete sessions expire and are cleaned up. The client asks which parts already exist rather than restarting from zero. Authorization is checked at session creation and finalization, and processing begins only after the immutable original object is complete.

S096 Search index misses recent posts [ ] [ ] [ ]

Situation: New posts appear in the database but are not searchable for several minutes.

"How would you diagnose and bound indexing delay?"

Answer: I would trace the path from database commit to outbox, broker, indexer, and search refresh, measuring lag at each boundary. A missing event, consumer backlog, poison document, or slow index refresh creates different remedies. The post remains authoritative in the database, and the product should communicate or tolerate the documented indexing delay. I would alert on oldest unindexed event, use idempotent upserts, and provide replay from a known offset. For critical read-after-write behavior, the UI can surface the newly created post directly without requiring immediate search consistency.

S097 Ranking feature service is unavailable [ ] [ ] [ ]

Situation: The online feature store used by the ranking model becomes unavailable during peak traffic.

"How should the feed degrade?"

Answer: Ranking should have a bounded feature-fetch timeout and a predefined fallback. I would use cached or default features, a simpler model with locally available signals, or chronological ranking rather than failing the entire feed. The response should still enforce privacy, blocking, and safety filters because those are correctness constraints, not optional ranking features. A circuit breaker prevents every request from waiting on the store, and fallback rate becomes an operational metric. After recovery, traffic ramps back gradually. Long term, I would replicate critical features or precompute them according to freshness requirements.

S098 Notification storm [ ] [ ] [ ]

Situation: A bug triggers millions of duplicate push notifications for the same event.

"How would you stop the storm and protect users?"

Answer: I would disable the producing feature or notification campaign immediately, pause affected consumers, and preserve event IDs for investigation. The sending path should enforce a deduplication key based on user, event, channel, and notification type, plus per-user and global rate limits. A kill switch must work independently of the faulty producer. I would distinguish queued but unsent messages from provider-accepted messages because some cannot be recalled. Recovery should discard confirmed duplicates and resume gradually. Tests should cover producer retries and consumer restarts so duplicate internal delivery never becomes duplicate user-visible delivery.

S099 Comments count and list disagree [ ] [ ] [ ]

Situation: A post displays 120 comments, but opening the list shows only 116 because some comments were deleted or moderation is delayed.

"How would you reason about the inconsistency?"

Answer: I would define what the count represents: all stored comments, visible comments for this viewer, or an approximate public aggregate. If deletions and moderation update the list and counter asynchronously, short disagreement may be acceptable, but the UI should not promise exactness. I would make counter updates idempotent, emit events for visibility changes, and run reconciliation against authoritative comment state. Viewer-specific filtering can mean different users legitimately see different counts, so a single global exact counter may be impossible. Product requirements determine whether to display an approximation, hide the count temporarily, or compute a costlier filtered value.

S100 Design an Instagram-like home feed under failure [ ] [ ] [ ]

Situation: You are asked to design a home feed that remains useful when ranking, notification, or analytics dependencies fail.

"What is your high-level design and graceful-degradation strategy?"

Answer: I would keep post creation durable in the primary store, publish changes through an outbox, and build feed candidates using hybrid fan-out: push for normal accounts and pull for celebrities. The read service merges candidates, applies visibility rules, ranks them, hydrates metadata in batches, and serves media through a CDN. Caches reduce repeated reads, while cursor pagination bounds work. Ranking has a chronological or cached fallback; analytics and notifications are asynchronous and cannot block reads. Timeouts, circuit breakers, idempotent consumers, lag metrics, and replay support contain failures. The critical promise is a safe usable feed, not that every enhancement is always fresh.

11. Coordination, Locks, and Distributed State

S101 Distributed lock expires during work [ ] [ ] [ ]

Situation: A worker holds a distributed lock, pauses for garbage collection, and continues writing after its lease has expired and another worker acquired it.

"Why is the lock not enough, and how would you protect the resource?"

Answer: The old worker cannot know that its lease expired while it was paused, so mutual exclusion at acquisition time is insufficient. I would attach a monotonically increasing fencing token to every lease and require the protected storage system to reject writes carrying an older token. The task should also be idempotent and periodically verify lease ownership, but that check alone has a race before the write. I would measure pause duration and renewal failures, bound task length, and test forced expiry. Correctness comes from the resource validating freshness, not from trusting the worker's local belief.

S102 Scheduled job runs on every replica [ ] [ ] [ ]

Situation: A nightly cleanup job is embedded in a service with ten replicas, and all ten execute it simultaneously.

"How would you ensure the job runs safely?"

Answer: I would first make the cleanup idempotent so duplicate execution cannot corrupt data. Then I would choose one ownership mechanism: a dedicated scheduler, a broker-delivered job with one consumer group, or leader election with a lease and fencing token. A simple in-memory flag cannot coordinate replicas. I would record a stable job-run ID, start and completion state, processed range, and retry history so interrupted work can resume. If the job can be partitioned safely, parallel workers may be better than a singleton, but ownership of each partition must still be explicit.

S103 Clock skew breaks expiry logic [ ] [ ] [ ]

Situation: Two servers disagree about time by 90 seconds, causing one to accept an expired token and another to reject a valid one.

"How would you design around clock differences?"

Answer: I would synchronize hosts with a reliable time service and alert on clock offset, but I would not assume clocks are perfectly equal. Authentication checks can allow a small bounded skew for issued-at and expiry claims while keeping token lifetimes short. For measuring durations and timeouts inside one process, I would use a monotonic clock rather than wall time. Distributed ordering should use versions, sequence numbers, or logical timestamps instead of comparing server clocks. If exact legal or financial timestamps matter, one authoritative service records them and downstream systems preserve that source value.

S104 Quorum cannot be reached [ ] [ ] [ ]

Situation: A replicated datastore requires a majority of nodes for writes, but a network partition leaves each side without enough votes.

"Would you lower the quorum to restore availability?"

Answer: Lowering quorum during a partition can allow conflicting writes and destroy the consistency guarantee the system was chosen to provide. I would confirm which nodes and links failed, restore connectivity or capacity, and keep the minority side read-only or unavailable for protected writes. If the product can tolerate weaker consistency, that should be a predesigned mode for specific data, not an emergency configuration change made under pressure. I would communicate the availability trade-off, preserve evidence, and test quorum-loss recovery. Critical state should fail clearly rather than accept writes that cannot later be reconciled safely.

S105 Lease renewal fails intermittently [ ] [ ] [ ]

Situation: A leader renews its lease through a coordination service, but brief network delays cause frequent leadership changes.

"How would you reduce flapping without hiding real failure?"

Answer: I would compare lease duration, renewal interval, observed network latency, process pauses, and coordination-service health. The leader should renew well before expiry and stop protected work when it cannot prove ownership. A modestly longer lease or retry window can absorb expected jitter, but it also delays failover, so I would choose it from measured worst-case behavior and recovery objectives. Fencing tokens protect against overlap if an old leader resumes. I would add jitter to elections, alert on churn, and avoid placing the coordination service behind overloaded dependencies that the leader itself controls.

S106 Concurrent username registration [ ] [ ] [ ]

Situation: Two requests attempt to register the same username at nearly the same time on different application instances.

"Where should uniqueness be enforced?"

Answer: I would treat the database uniqueness constraint as the final authority because application-level check-then-insert logic races across instances. Each request can perform an early availability check for user experience, but the insert must still handle a uniqueness violation and return a clear conflict. Normalization rules such as case folding must be identical in lookup and constraint keys. If usernames are sharded, all variants of one normalized name must route to the same owner or use a dedicated reservation service. Failed or abandoned reservations need explicit expiry so names are not lost indefinitely.

S107 Inventory reservation expires [ ] [ ] [ ]

Situation: Checkout reserves the last item, payment takes several minutes, and the reservation expires just before payment succeeds.

"How would you prevent selling unavailable inventory?"

Answer: I would model reservation as an explicit state with an owner, quantity, expiry, and version rather than decrementing and hoping payment finishes. Payment completion must atomically verify that the reservation is still valid before converting it to a sale. If it expired, the workflow should void or refund the payment and notify the user instead of creating negative inventory. A sweeper releases abandoned reservations idempotently. The reservation duration balances conversion against stock utilization, and the UI should show the deadline. Reconciliation compares inventory, reservations, orders, and provider payments to repair rare boundary failures.

S108 Lock service is unavailable [ ] [ ] [ ]

Situation: A critical workflow depends on a distributed lock service that becomes unavailable.

"Should the application continue without the lock?"

Answer: If the lock protects correctness, silently continuing is unsafe because multiple workers may modify the same resource. I would fail closed for those operations, return a retryable status, and preserve read-only or unrelated functions. Existing lease holders must obey expiry and fencing rules; local cached ownership is not proof. If product requirements demand availability, the workflow needs a different design, such as partitioned ownership, optimistic concurrency, or idempotent operations, decided before the outage. I would restore the coordination quorum, limit retries, and verify no overlapping work occurred before resuming full traffic.

S109 Consensus writes are too slow [ ] [ ] [ ]

Situation: A strongly consistent replicated store meets correctness needs but write latency is too high for every user interaction.

"How would you reduce latency without discarding correctness everywhere?"

Answer: I would separate data by consistency requirement. Account ownership, payments, and uniqueness may still need quorum writes, while analytics, counters, feed ranking, and presence can use asynchronous or local updates. I would also inspect replica placement, network latency, batching, payload size, and whether every transaction performs unnecessary consensus rounds. Moving a quorum closer improves latency for one region but changes disaster tolerance. The goal is not to weaken the entire database setting; it is to keep strong coordination only on the small state transitions whose correctness justifies the latency and availability cost.

S110 User moves between replicas and sees older data [ ] [ ] [ ]

Situation: A user reads a new profile value from one replica, then a later request reaches a lagging replica and shows the previous value.

"How would you provide monotonic reads?"

Answer: I would return or retain a data version from the first read and require subsequent reads for that session to use a replica that has applied at least that version. A simpler option is temporary session affinity, though failover can break it and unevenly distribute load. For recent writes, routing the user to the primary for a bounded period may provide both read-your-writes and monotonic behavior. I would apply this only where the product needs it because version-aware routing increases complexity. Replication-lag metrics and a defined fallback determine whether to wait, switch replicas, or use the primary.

12. Service Discovery, Routing, and Gateways

S111 Service discovery returns dead instances [ ] [ ] [ ]

Situation: Clients continue sending traffic to instances that terminated several minutes ago.

"Where would you investigate the stale routing information?"

Answer: I would trace registration, health reporting, discovery propagation, client caching, and load-balancer removal separately. The registry may be correct while a client keeps an old endpoint list beyond its TTL, or shutdown may not deregister before termination. I would shorten unsafe cache periods, add active health checks, and make clients retry another endpoint only within a bounded request deadline. Graceful shutdown should fail readiness before process exit. Discovery data can be briefly stale, so connection failures must be expected. I would alert on traffic to terminated instance IDs and measure removal time end to end.

S112 DNS change does not take effect [ ] [ ] [ ]

Situation: Traffic should move to a new endpoint, but some clients keep using the old address for hours.

"Why can DNS-based failover be slow?"

Answer: DNS records have TTLs, but resolvers, operating systems, JVMs, proxies, and applications may cache answers differently or ignore the intended period. Existing TCP connections also continue using the old address after a fresh lookup would change. I would inspect effective client caching and connection lifetimes, keep the old endpoint healthy during transition, and reduce TTL well before planned migration. Emergency DNS changes are not instantaneous. For faster failover, a stable anycast, load-balancer, or service-mesh address can move backends without depending on every client refreshing DNS at the same moment.

S113 API gateway becomes the bottleneck [ ] [ ] [ ]

Situation: Backend services have spare capacity, but all requests slow down at the shared API gateway.

"How would you confirm and remove the bottleneck?"

Answer: I would compare gateway connection count, CPU, memory, event-loop or worker saturation, TLS cost, request buffering, authentication latency, logging, and upstream timing. Traces should show whether delay occurs before routing or in a backend. Immediate containment may scale gateway instances, disable expensive optional filters, reduce verbose logging, and cap large requests. I would preserve rate limiting and authorization even in degraded mode. Long term, I would benchmark each plugin, distribute gateways across failure domains, keep routing stateless, and ensure gateway autoscaling uses concurrency and connection metrics rather than backend CPU.

S114 Health check marks unhealthy service as healthy [ ] [ ] [ ]

Situation: A load balancer checks `/health`, which returns 200 even when the service cannot reach its database.

"How would you redesign health checks?"

Answer: I would separate liveness from readiness. Liveness answers whether the process should be restarted; readiness answers whether it can currently serve routed traffic. Readiness should cover the minimum critical dependencies for that endpoint class, using short cached checks or real request signals so health probing does not overload the database. It should use hysteresis to avoid flapping during brief dependency errors. Some endpoints may remain useful without the database, so separate pools or degraded routing can preserve them. A shallow process check alone creates false success and keeps known-bad instances in rotation.

S115 Proxy and application both retry [ ] [ ] [ ]

Situation: A proxy retries each failed request twice, and the application client also retries twice, producing up to nine downstream attempts.

"How would you control retry amplification?"

Answer: I would assign retry ownership to one layer for each dependency path and enforce a shared end-to-end deadline. Intermediate proxies should usually avoid retrying non-idempotent operations and expose attempt metadata so hidden retries are visible. The chosen retry layer uses a small budget, exponential backoff, jitter, and idempotency for writes. I would inspect traces for repeated attempt IDs and calculate worst-case multiplication across the call graph. Circuit breakers and concurrency limits protect the dependency during failure. A successful retry rate is not automatically good if those retries consume the remaining capacity.

S116 Load balancer sends traffic during shutdown [ ] [ ] [ ]

Situation: A service fails readiness, but new requests continue arriving for 20 seconds while the load balancer updates.

"How would you make deployments safe during this propagation delay?"

Answer: The shutdown sequence must account for the load balancer's removal delay. I would first mark the instance unready, wait longer than observed propagation while still accepting in-flight traffic, then stop accepting new work and drain existing requests within the platform grace period. The process should not close listeners immediately after readiness changes. Long-lived streams need explicit migration or reconnect behavior. I would measure the actual last-new-request time during deployment and align pre-stop hooks, deregistration delay, and termination timeout. Hard shutdown remains the final bound for stuck requests.

S117 Choosing gRPC or REST between services [ ] [ ] [ ]

Situation: A team proposes replacing all internal REST calls with gRPC to improve performance.

"How would you decide whether the migration is justified?"

Answer: I would measure current serialization, network, and application time before assuming protocol overhead is the bottleneck. gRPC provides typed contracts, efficient binary messages, streaming, and code generation, but adds tooling, proxy compatibility, debugging, and browser-boundary considerations. REST remains simple and widely observable for resource-oriented calls. I might choose gRPC for high-volume low-latency internal communication or streaming while retaining REST at public boundaries. The migration should include deadlines, error mapping, version compatibility, load tests, and incremental rollout. Changing protocol does not fix slow database queries or poorly bounded payloads.

S118 Webhook traffic arrives in bursts [ ] [ ] [ ]

Situation: An external provider sends hundreds of thousands of callbacks immediately after recovering from its own outage.

"How would you protect the webhook endpoint and downstream services?"

Answer: I would authenticate requests, enforce payload and rate limits, and acknowledge quickly only after durably storing each callback. Slow business processing moves to a queue, where idempotent consumers absorb the burst at controlled concurrency. Provider event IDs prevent duplicate effects, and timestamp checks reject stale replay attacks according to policy. If ingress capacity is threatened, I would coordinate provider backoff or return a retryable response while preserving enough headroom for normal traffic. Queue age, storage growth, and signature failures need alerts. Recovery drains gradually so the burst is not merely shifted downstream.

S119 Tenant routing sends requests to the wrong database [ ] [ ] [ ]

Situation: A routing-cache bug maps a tenant to another tenant's database shard.

"How would you contain the incident and harden routing?"

Answer: I would stop affected operations, invalidate the routing cache, and verify whether any cross-tenant reads or writes occurred using audit logs. Routing alone must not be the only isolation boundary: each database query or credential should also be scoped to the authenticated tenant. I would store shard mappings in a versioned authoritative directory, include tenant identity in connection selection, and reject mismatches at the database or repository layer. Updates require atomic version changes and safe cache invalidation. Cross-tenant integration tests and canary routing changes should attempt the exact failure rather than checking only successful tenants.

S120 Global traffic manager routes to the nearest region [ ] [ ] [ ]

Situation: A latency-based global router sends users to the nearest region, but that region's database is overloaded.

"Should proximity remain the only routing signal?"

Answer: No. Routing should consider regional health, capacity, dependency readiness, data residency, and session consistency in addition to network latency. I would reduce traffic to the overloaded region gradually while confirming alternate regions have capacity and access to sufficiently fresh data. Sticky sessions may need controlled breakage or version-aware reads. A global router should use stable health signals with hysteresis so it does not oscillate traffic between regions. I would test evacuation regularly and reserve headroom; nearest-region routing is valuable only while that region can serve the complete request safely.

13. Storage Systems and Data Lifecycle

S121 Object upload succeeds but metadata write fails [ ] [ ] [ ]

Situation: A photo reaches object storage, but the database transaction that records its owner and status fails.

"How would you prevent permanent orphaned objects?"

Answer: I would upload under a temporary session key and finalize visibility only after the metadata transaction commits. A durable workflow records upload intent, object checksum, owner, and expiry. If metadata fails, the object remains private and a cleanup job deletes abandoned temporary objects after a safe window. If metadata commits but finalization fails, an idempotent retry completes it. Direct client uploads should use narrowly scoped credentials tied to the session. Reconciliation compares temporary objects with upload records so neither storage listing nor database state alone is trusted as complete truth.

S122 Storage volume reaches capacity [ ] [ ] [ ]

Situation: A stateful service reports 95 percent disk usage and write latency is rising.

"What would you do before the disk becomes full?"

Answer: I would identify what is growing, the growth rate, safe deletion candidates, replication status, and how the storage engine behaves near full capacity. Immediate actions may add capacity, throttle writes, pause compaction or backfills if they consume temporary space, and remove only verified disposable data. Deleting active files manually can corrupt the store. I would preserve enough free space for recovery, snapshots, and compaction. Long term, I would alert on time-to-full rather than a fixed percentage, enforce retention, test expansion, and keep emergency headroom because many databases fail before reaching exactly 100 percent.

S123 Compaction causes latency spikes [ ] [ ] [ ]

Situation: An LSM-based datastore has predictable latency spikes whenever background compaction runs.

"How would you reduce user impact?"

Answer: I would correlate compaction throughput, pending bytes, disk bandwidth, CPU, and write amplification with request latency. Compaction is necessary to merge files and reclaim space, so disabling it indefinitely only postpones a larger failure. I would tune compaction concurrency and rate limits, provide faster or separate storage bandwidth, reduce excessive small writes, and schedule heavy maintenance away from peak load where possible. Admission control protects foreground requests when debt is high. I would alert on compaction backlog and test recovery after bursts, because a system that meets peak writes but never catches up remains unstable.

S124 Millions of records expire simultaneously [ ] [ ] [ ]

Situation: A retention policy gives every session the same midnight expiry, producing a large deletion spike.

"How would you smooth the workload?"

Answer: I would avoid synchronized physical deletion. Expiry times can include jitter, and logical expiration should make records invisible immediately while background workers delete them in bounded batches. Partitioning by time may let the database drop an old partition more efficiently than deleting individual rows. I would rate-limit cleanup according to database headroom and monitor tombstones, storage reclamation, and replication lag. If exact midnight invalidation is required for correctness, reads still enforce that timestamp, but physical cleanup can occur later. The retention contract and the storage-reclamation schedule are related but do not need to be identical.

S125 Moving old media to cold storage [ ] [ ] [ ]

Situation: Most photos older than two years are rarely accessed, and storage cost is growing quickly.

"How would you introduce archival storage without breaking reads?"

Answer: I would measure access frequency, restore latency tolerance, retrieval cost, and legal retention before choosing an archive tier. Metadata keeps the object's current storage class and state, while a lifecycle job moves eligible immutable objects and records completion idempotently. Reads either restore asynchronously with a processing status or use a tier that supports acceptable direct retrieval. Popular or recently requested objects can be promoted again. I would verify checksums after movement, preserve redundancy, and test deletion separately. Cost estimates must include retrieval and egress, not only the lower monthly storage price.

S126 Silent data corruption is suspected [ ] [ ] [ ]

Situation: A media file exists and returns HTTP 200, but users report that some images are visibly corrupted.

"How would you detect and repair corruption?"

Answer: I would compare the stored object's checksum with the checksum recorded at successful upload and with replicas or backups. HTTP success proves retrieval, not integrity. Affected objects should be quarantined from normal delivery while the system attempts repair from a known-good copy or regenerates derived variants from the original. I would inspect whether corruption occurred during upload, processing, storage, or transfer by validating checksums at each boundary. Background scrubbing can detect dormant corruption before access. Redundancy helps only when replicas are independently verified and bad data is not automatically copied over good data.

S127 Append-only log grows without bound [ ] [ ] [ ]

Situation: An audit or event log is retained forever on expensive primary storage.

"How would you control growth without losing required evidence?"

Answer: I would classify records by legal, security, replay, and analytical value and assign explicit retention periods. Recent data can remain on fast searchable storage, while older immutable segments move to compressed archive storage with an index for discovery. Deletion must respect holds and be auditable. If the log supports replay, snapshots or materialized checkpoints can reduce how much history consumers need online. I would monitor daily growth, compression ratio, and restore time. Keeping everything forever is not automatically safer; it increases cost, privacy exposure, and the amount of data that must be protected.

S128 Blob and metadata become inconsistent [ ] [ ] [ ]

Situation: The database says a video is ready, but the referenced object does not exist in storage.

"How would you make publication reliable?"

Answer: I would separate upload, processing, and published states. The system publishes the metadata reference only after verifying the final object and checksum. Since the database and object store do not share a transaction, a durable workflow or outbox tracks each transition and retries idempotently. Readers treat only `READY` objects as available and use a fallback for missing media. Reconciliation scans database references and storage manifests to find missing or orphaned objects. The fix should also identify whether deletion, lifecycle rules, or failed copy operations removed the object after publication, because each needs different prevention.

S129 Time-series metrics database is overloaded [ ] [ ] [ ]

Situation: Monitoring storage cannot keep up because every request includes user ID, URL, and request ID as metric labels.

"Why is label cardinality dangerous?"

Answer: Each unique label combination creates a separate time series, so unbounded values such as user or request IDs can multiply memory, index, storage, and query cost. I would remove high-cardinality dimensions from metrics and keep them in sampled logs or traces. Metrics should use bounded labels such as endpoint template, status class, region, and service version. During the incident, I would drop or relabel the offending series at ingestion and protect the monitoring system from further growth. Cardinality budgets and linting should block unsafe instrumentation before deployment, because observability must not become a production dependency failure.

S130 Data retention conflicts with analytics needs [ ] [ ] [ ]

Situation: Privacy policy requires deleting raw user events after 30 days, but analysts want year-over-year trends.

"How would you preserve useful insight while respecting retention?"

Answer: I would identify which long-term questions need aggregates rather than individual events. Before raw data expires, a governed pipeline can produce sufficiently coarse, non-identifying summaries with documented dimensions and minimum group sizes. The raw events and direct identifiers are then deleted on schedule across primary, derived, and backup systems according to policy. I would prevent analysts from reconstructing individuals through overly detailed combinations and review whether the aggregates still count as personal data. Retention enforcement, lineage, and deletion evidence should be automated so an analytical convenience does not silently override the approved privacy boundary.

14. Stream Processing and Analytics

S131 Late events change completed windows [ ] [ ] [ ]

Situation: Mobile clients upload activity hours late, after an hourly analytics window has already been published.

"How would you handle late-arriving events?"

Answer: I would process by event time and define an allowed lateness using watermarks rather than assuming arrival time equals occurrence time. Windows remain revisable until the watermark passes; later events can update results, enter a correction stream, or be dropped according to product requirements. Every event needs a stable ID for deduplication. Dashboards should indicate whether values are preliminary or final. A very long lateness allowance improves completeness but increases retained state and delays finality, so I would choose it from observed client behavior and the business value of correcting old results.

S132 Analytics pipeline double-counts events [ ] [ ] [ ]

Situation: A stream processor restarts and some engagement events appear twice in daily totals.

"How would you make the aggregation accurate?"

Answer: I would determine whether duplicates originate at production, broker redelivery, replay, or state recovery. Events need stable IDs, and the processor can deduplicate within a bounded window or write idempotent upserts keyed by event and aggregation bucket. Exactly-once framework guarantees help only within their supported sources, state, and sinks; an external database may still duplicate a nontransactional write. I would checkpoint offsets and aggregation state together, then reconcile daily totals against an authoritative raw-event set. The acceptable deduplication window and storage cost depend on how late duplicates can arrive.

S133 One stream partition is far behind [ ] [ ] [ ]

Situation: Most Kafka partitions have no lag, but one partition is millions of events behind.

"What would you investigate before adding consumers?"

Answer: I would compare event rate, key distribution, record size, processing errors, and downstream calls for the lagging partition. One hot key or poison sequence can serialize all work there, and additional consumers cannot process the same partition concurrently within one group. Immediate containment may isolate the problematic key, increase processing efficiency, or temporarily route non-order-sensitive work through more partitions. Repartitioning changes ordering and requires a migration plan. I would monitor per-partition lag and oldest-event age, because cluster-wide averages hide the exact hotspot that controls completion time.

S134 Real-time count changes after correction [ ] [ ] [ ]

Situation: A dashboard shows 10,000 active users, then later revises the number downward after duplicate removal.

"How would you explain and design for changing results?"

Answer: Real-time results often trade final accuracy for speed. I would define whether the dashboard is approximate, preliminary, or final and show the freshness time. Stable event IDs and bounded deduplication reduce corrections, while a later batch reconciliation can publish an authoritative number. Consumers must accept upserts or retractions rather than only additive counts. If users require immutable numbers for billing, I would not use the provisional stream result as the invoice source. The system should expose the semantic difference clearly so a corrected metric is not mistaken for data loss or an unexplained outage.

S135 Backfill overlaps live processing [ ] [ ] [ ]

Situation: A historical replay runs while the live pipeline continues, and both write the same daily aggregates.

"How would you avoid double-counting or overwriting newer results?"

Answer: I would isolate the backfill using a separate run ID, output table, or versioned namespace rather than writing directly into live aggregates. The backfill reads a fixed input range and produces deterministic results, which are validated before an atomic pointer or partition swap. If both must write one sink, idempotent keys include the window and version, and a coordinator defines which result wins. Live events after the backfill boundary remain separate. I would document source offsets and timestamps so the team can reproduce the exact dataset and safely resume an interrupted run.

S136 Event schema changes break consumers [ ] [ ] [ ]

Situation: A producer renames a required field, and several independent consumers fail after deployment.

"How would you evolve event contracts safely?"

Answer: I would restore compatibility, then treat event schemas as long-lived contracts. Changes should be additive: introduce the new field, keep the old field during migration, and let consumers tolerate unknown fields and sensible defaults. A schema registry or compatibility check can block breaking publication. Consumers migrate and expose usage before the producer removes deprecated data. For meaning changes, I would use a new field or event version rather than reusing the old name. Historical replay must remain possible, so deserializers and transformation tools need to understand older schemas, not only today's producer output.

S137 Event time and processing time disagree [ ] [ ] [ ]

Situation: A purchase occurred at 11:59 PM but arrived at the server at 12:03 AM.

"Which day should receive the event?"

Answer: The answer depends on the business definition. User-behavior analytics may group by event time, while operational load metrics use processing time, and financial records may use the authoritative server acceptance time. I would store both timestamps plus their source and timezone, validate unreasonable client clocks, and document which field each calculation uses. Event-time windows need lateness handling and possible corrections. I would not silently use the device timestamp for money or compliance decisions because users can change it. One event can legitimately belong to different dates in different reports when their semantics differ.

S138 Deduplication state becomes too large [ ] [ ] [ ]

Situation: A consumer stores every processed event ID forever to prevent duplicates, and the deduplication table is now enormous.

"How would you bound the state?"

Answer: I would determine the maximum realistic redelivery or replay window and retain deduplication IDs only for that period, partitioned by time for efficient expiry. If business effects must never repeat beyond that window, the durable entity state should enforce a uniqueness key instead of relying on a global event-history table. Bloom filters can reduce lookups but have false positives and cannot be the sole correctness mechanism. I would separate short-term transport deduplication from permanent business invariants. Metrics on duplicate age reveal whether the chosen window is adequate before old IDs are removed.

S139 Dashboard is ten minutes behind [ ] [ ] [ ]

Situation: Executives expect near-real-time metrics, but the dashboard consistently lags by ten minutes.

"How would you find where freshness is lost?"

Answer: I would attach timestamps at event creation, broker ingestion, processor completion, warehouse load, query materialization, and dashboard refresh, then calculate lag at each boundary. The bottleneck could be consumer backlog, micro-batch cadence, warehouse commit time, cached queries, or the UI refresh interval. I would fix the dominant stage rather than making every interval smaller, which can increase cost and contention. The dashboard should display its data-through time. A freshness service-level objective and oldest-event-age alerts make delay visible before an executive notices a stale chart.

S140 Stateful stream processor loses a node [ ] [ ] [ ]

Situation: A stream-processing worker crashes while holding local aggregation state for several partitions.

"How should processing recover without losing or duplicating results?"

Answer: State and input progress must be checkpointed consistently to durable storage. On failure, another worker restores the latest checkpoint, reclaims the partitions, and replays events after the stored offsets. Outputs need transactional or idempotent semantics because replay repeats work after the checkpoint. I would monitor checkpoint age, duration, and restore time and keep enough broker retention to cover the recovery window. Very large state may require incremental checkpoints or a remote state store. A successful worker restart is not proof of correctness; reconciliation should compare recovered aggregates with source events for a sampled range.

15. Search, Ranking, and Machine Learning Systems

S141 Search suddenly returns zero results [ ] [ ] [ ]

Situation: Search traffic is normal, but result count drops to zero for many common queries after an index deployment.

"How would you contain and diagnose the issue?"

Answer: I would compare the new and previous index aliases, document counts, shard health, query analyzer versions, filters, and sample query plans. If the new index is clearly wrong, I would atomically point traffic back to the previous verified index rather than attempting in-place repair. The source database remains authoritative, so I would check whether indexing failed or querying changed. I would preserve the broken index for diagnosis, then rebuild and validate counts, fields, and representative queries before switching. Canary queries and minimum-result guardrails should block future index promotion when common searches collapse.

S142 Rebuilding a search index without downtime [ ] [ ] [ ]

Situation: A schema change requires reindexing hundreds of millions of documents.

"How would you migrate traffic safely?"

Answer: I would build a new versioned index from a consistent snapshot while continuing to capture live changes through an event stream or change log. After the bulk load, the new index catches up to a recorded boundary. I would validate document counts, sampled content, query relevance, latency, and permission filters, then send a small canary percentage or shadow queries before atomically switching an alias. The old index remains available for rollback until confidence is high. Writers should not dual-write two indexes without a repair strategy, because partial failures can silently make them diverge.

S143 Autocomplete service is overwhelmed by trending queries [ ] [ ] [ ]

Situation: A breaking-news event causes millions of users to type the same prefixes, saturating autocomplete servers.

"How would you serve suggestions efficiently?"

Answer: I would cache popular prefix results at the edge or application layer and coalesce concurrent misses so one computation serves many requests. Suggestions can be precomputed from a trie or search index and refreshed asynchronously, with short TTLs for trends and longer TTLs for stable prefixes. The client should debounce keystrokes and cancel obsolete requests. Rate limits protect against automation. If the ranking service fails, a cached or static popular list is an acceptable fallback. I would monitor top prefixes and cache concentration because the same popularity that creates load also makes caching highly effective.

S144 Model rollout harms one user group [ ] [ ] [ ]

Situation: Overall engagement improves after a ranking-model launch, but a regional subgroup sees much worse results.

"How would you detect and respond to this regression?"

Answer: Aggregate success can hide segment harm, so I would monitor quality, latency, safety, and retention across predeclared regions, languages, device classes, and new-user cohorts while protecting privacy. I would pause or roll back the model for the affected group using a feature flag, then compare feature availability, training representation, and serving behavior. Statistical significance matters, but severe safety or usability harm does not require waiting for a perfect experiment. Future launches need segment guardrails and minimum sample checks. The durable fix may require better data, features, calibration, or a separate model for that context.

S145 Online features differ from training features [ ] [ ] [ ]

Situation: A model performs well offline but poorly in production because feature values are calculated differently at serving time.

"How would you prevent training-serving skew?"

Answer: I would define features once and reuse the same transformation logic or managed feature definitions for offline training and online serving. Each prediction should record feature versions and freshness so bad outcomes can be reproduced. I would compare sampled online vectors with offline recomputation and alert on distribution drift, missing values, and default rates. Backfills must use event-time-correct data to avoid leakage from the future. A feature store can help but does not guarantee consistency if teams bypass it. Model rollout should include shadow predictions against production traffic before decisions affect users.

S146 Vector search latency grows with corpus size [ ] [ ] [ ]

Situation: Semantic search was fast at one million embeddings but exceeds the latency budget at one hundred million.

"What trade-offs would you evaluate?"

Answer: I would measure index build, memory, recall, query latency, filter selectivity, and shard distribution. Approximate nearest-neighbor indexes trade some recall for speed and require tuning parameters such as search breadth. I might partition by a safe coarse attribute, use a smaller candidate index, compress vectors, or retrieve candidates semantically and rerank a limited set. Filters must be applied without destroying index efficiency or leaking unauthorized items. More replicas improve read capacity but not one-query complexity. I would choose settings using relevance evaluation and a latency target rather than optimizing benchmark recall in isolation.

S147 New user has no ranking history [ ] [ ] [ ]

Situation: A newly registered user has no follows, clicks, or watch history, so personalization has little signal.

"How would you handle the cold-start problem?"

Answer: I would use signals available immediately: selected interests, locale, language, device context, broadly popular content, and editorial or safety constraints. The onboarding flow can ask for a small number of high-value preferences without creating excessive friction. Early interactions update the profile quickly, while exploration intentionally shows diverse candidates to learn interests. A chronological or regional feed is a clear fallback. I would measure time to useful personalization and avoid overfitting to the first click. New-user ranking must still enforce privacy and quality; lack of history is not permission to show risky content.

S148 A/B test groups become contaminated [ ] [ ] [ ]

Situation: Users switch devices and sometimes receive both control and treatment behavior during one experiment.

"How would you make experiment assignment stable?"

Answer: I would choose a durable assignment unit aligned with the hypothesis, usually authenticated user ID, and compute treatment deterministically from experiment and subject IDs. Anonymous users may use a device or cookie key, but the migration after login must be defined. Assignment should happen in one service or shared library and be logged with exposure, not merely eligibility. Cross-service flags must use the same value. I would exclude bots and internal traffic where appropriate and analyze contamination explicitly. Stable assignment prevents carryover effects from being mistaken for treatment impact.

S149 Safety model exceeds latency budget [ ] [ ] [ ]

Situation: A content-safety classifier adds 800 ms to every post upload and occasionally times out.

"Would you make moderation asynchronous?"

Answer: I would separate fast mandatory checks from slower deep analysis based on risk. The upload can remain quarantined or visible only to the owner until required moderation completes, then publish after an asynchronous classifier decision. For lower-risk edits, a fast model may allow provisional visibility with later review, but unsafe exposure must match policy. Timeouts should fail to the safer state, not automatically approve. I would batch GPU work, cache identical media fingerprints, and monitor queue age. The product must show processing status so safety latency does not appear as a mysterious failed upload.

S150 Search must handle spelling mistakes [ ] [ ] [ ]

Situation: Users frequently misspell names, but aggressive fuzzy matching returns irrelevant or unsafe results.

"How would you balance recall and precision?"

Answer: I would normalize case and accents, use prefix and edit-distance matching with limits based on query length, and rank exact or popular matches above fuzzy ones. Candidate generation and final ranking should be separate so the system can retrieve broadly but apply language, popularity, safety, and authorization filters before display. Very short queries need stricter rules because one edit changes meaning significantly. I would evaluate with real anonymized query-result judgments, including languages and names, and expose a correction hint rather than silently rewriting every query. Zero-result rate alone is not a sufficient success metric.

16. Networking, Protocols, and Connections

S151 DNS resolution failures spike [ ] [ ] [ ]

Situation: Application instances are healthy, but outbound calls fail because service hostnames intermittently cannot be resolved.

"How would you diagnose and contain a DNS dependency failure?"

Answer: I would compare resolver error codes, latency, cache hit rate, affected zones, and whether failures occur at the host, container, or upstream DNS service. Existing connections may still work, so connection reuse can preserve limited traffic while new lookups fail. I would avoid tight application retries that overload resolvers and use bounded caching according to record TTL and failure policy. If a backup resolver is approved, failover should be tested rather than improvised. Long term, I would monitor DNS as a real dependency, distribute resolvers across failure domains, and capacity-test them during instance churn.

S152 TLS certificate is about to expire [ ] [ ] [ ]

Situation: Monitoring shows that a public API certificate expires in 18 hours, and automatic renewal has failed.

"What would you do, and how would you prevent recurrence?"

Answer: I would confirm the certificate, domain coverage, serving endpoints, and renewal failure, then use the approved emergency issuance path before making unrelated changes. Deployment must include the complete chain and preserve private-key security. I would test the new certificate from multiple networks and verify old connections and all load balancers have rotated. Prevention includes automated renewal with enough lead time, alerts at several thresholds, ownership, inventory of every endpoint, and a safe manual fallback. A successful certificate file update is not sufficient until the externally served certificate and handshake are verified.

S153 Connection pool contains stale sockets [ ] [ ] [ ]

Situation: After a database failover, many pooled connections point to the old primary and requests fail until instances restart.

"How would you make the pool recover automatically?"

Answer: I would configure connection validation, maximum lifetime, and failure-based eviction so broken sockets leave the pool quickly. The database endpoint or driver must support primary discovery or a stable proxy, and the application should open new connections gradually to avoid a reconnection storm. Short bounded retries can hide a transient failover only for idempotent operations. I would inspect DNS caching and driver topology refresh because a pool cannot recover if new connections still target the old address. A failover test should verify detection time, transaction errors, pool recovery, and the absence of duplicate writes.

S154 Large packets disappear on one network path [ ] [ ] [ ]

Situation: Small API requests succeed, but large responses hang only for users behind a particular network provider.

"What network problem would you suspect and how would you verify it?"

Answer: I would suspect an MTU or path-MTU-discovery problem where fragmented or oversized packets are dropped, especially if required ICMP messages are blocked. I would compare packet captures, response sizes, TCP retransmissions, and tests with smaller maximum segment sizes across affected and healthy paths. Compression or smaller responses may contain impact, but the network path or tunnel configuration needs correction. I would also rule out proxy body limits and timeouts because they create similar size-dependent symptoms. Monitoring by payload size and provider can reveal a failure hidden by overall success metrics.

S155 HTTP/2 connection carries many slow streams [ ] [ ] [ ]

Situation: Hundreds of requests share one HTTP/2 connection, and packet loss causes several otherwise independent streams to slow down.

"Why can multiplexing still suffer shared latency?"

Answer: HTTP/2 removes application-layer request ordering on one connection, but all streams still share the same TCP loss recovery and congestion window. A lost packet can delay bytes needed by multiple streams. I would verify retransmissions, connection concentration, and whether a few connections carry excessive traffic. More connections or HTTP/3 over QUIC may reduce transport-level head-of-line effects, but add operational and compatibility considerations. I would first fix packet loss and avoid assuming a protocol switch solves an unhealthy network. Load tests should include loss and latency, not only a clean high-bandwidth link.

S156 Large responses saturate network bandwidth [ ] [ ] [ ]

Situation: CPU and database metrics are healthy, but service latency rises as outbound network throughput reaches its limit.

"How would you reduce the bandwidth bottleneck?"

Answer: I would identify endpoints and fields responsible for bytes, then reduce payloads through pagination, bounded summaries, appropriate compression, and avoiding repeated metadata. Static media should move to object storage and a CDN rather than passing through application servers. Compression trades CPU for bandwidth, so I would measure both and avoid recompressing already compressed formats. Streaming can reduce memory and time to first byte but not total bytes. I would set response-size budgets, monitor percentile payload size, and scale network capacity only after removing unnecessary transfer. Client caching can eliminate repeated downloads when freshness permits.

S157 NAT port exhaustion [ ] [ ] [ ]

Situation: Thousands of containers share a small set of outbound addresses and begin failing new connections to external APIs.

"How would you recognize and fix source-port exhaustion?"

Answer: I would look for connection failures despite healthy destinations, high translation-table usage, many short-lived outbound connections, and sockets in `TIME_WAIT`. Immediate mitigation includes connection pooling and keep-alive, reducing retry churn, spreading traffic across more egress addresses, and raising safe platform limits. I would verify that clients actually reuse connections and that idle settings do not close them too aggressively. Long term, I would capacity-plan ports from concurrent destinations and connection lifetime, monitor NAT utilization, and avoid routing unrelated high-volume workloads through one small egress boundary.

S158 Chat messages are slow across regions [ ] [ ] [ ]

Situation: Users in Asia send chat messages through a primary service in North America and experience noticeable delay.

"How would you reduce latency while preserving conversation order?"

Answer: I would place connection gateways near users but assign each conversation or participant pair an authoritative ordering owner. Regional gateways accept authenticated messages, attach stable IDs, and forward them over persistent links to that owner, which assigns sequence numbers and durably stores the message. Delivery fans out from nearby regions. Moving ownership closer can reduce latency but requires a controlled handoff so two regions do not assign overlapping order. I would define whether total order is needed per conversation or only per sender. Offline delivery and deduplication must use IDs rather than connection state.

S159 WebSocket fleet needs to scale [ ] [ ] [ ]

Situation: Millions of clients maintain long-lived WebSocket connections, and rolling deployments disconnect too many users at once.

"How would you operate this connection-heavy service?"

Answer: I would separate connection gateways from durable messaging state and route messages through a broker or ownership directory rather than storing important state only in process memory. Autoscaling should use connection count, memory, and event-loop lag, not request rate. Deployments drain a small percentage of gateways at a time, stop new connections, and give clients randomized reconnect backoff. Load balancers need suitable idle timeouts and health checks. Presence is eventually consistent and rebuilt after disconnects. Capacity planning includes reconnect storms because a regional restart can create more load than normal steady-state traffic.

S160 Proxy timeout is shorter than application timeout [ ] [ ] [ ]

Situation: The application allows a request to run for 60 seconds, but the gateway closes the client connection after 30 seconds.

"What problems does this create and how would you align timeouts?"

Answer: The application may continue expensive work after the caller has already received a timeout, and the caller may retry, producing duplicate load or writes. I would set an end-to-end deadline and propagate the remaining budget downstream. Inner timeouts should be shorter than outer timeouts so failures return while the caller can still respond. Cancellation should stop work where safe, while writes use idempotency because cancellation cannot prove nothing committed. I would document timeout ownership across client, gateway, service, and database and test the total worst case including retries rather than configuring each layer independently.

17. Multi-Region Systems and Global Consistency

S161 Active-active regions update the same profile [ ] [ ] [ ]

Situation: A user changes their biography in two regions during a network partition, creating conflicting values.

"How would you resolve the conflict?"

Answer: I would first decide whether last-write-wins is acceptable for this field and which timestamp can be trusted. For profile text, a versioned last-writer policy or user-visible conflict may be reasonable; for money or permissions it would not be. Regions should attach versions and preserve both conflicting operations until resolution instead of silently discarding one. A home-region owner can avoid most conflicts at the cost of cross-region latency and reduced partition availability. I would choose conflict semantics per data type, test partitions, and expose repair metrics because replication technology cannot invent the business rule.

S162 Region-specific ID generators collide [ ] [ ] [ ]

Situation: Two regions generate the same numeric order ID after operating independently during a partition.

"How would you generate globally unique IDs without central coordination on every request?"

Answer: I would use an ID scheme that encodes enough independent entropy or ownership, such as random UUIDs or time-based IDs containing a region or worker component. If IDs must be roughly ordered, clock rollback and worker-ID assignment need protection. Allocating disjoint numeric ranges per region is another option but requires range management. A central sequence guarantees order but becomes a latency and availability dependency. I would distinguish uniqueness from gap-free or globally ordered requirements because the latter are far more expensive and rarely necessary for external identifiers.

S163 Global username uniqueness [ ] [ ] [ ]

Situation: Users can register in multiple regions, but each username must be globally unique.

"What consistency trade-off would you make?"

Answer: Global uniqueness requires one serialization point per normalized username. I could route all reservations through a globally consistent store or deterministically assign each name to a home partition, accepting cross-region latency and reduced availability during certain partitions. A purely local active-active write can create duplicates that need painful user-visible conflict resolution. I would use a short reservation followed by account creation, with idempotent completion and expiry for abandoned claims. Availability is preserved for browsing and unrelated actions, while this narrow operation pays the coordination cost because duplicate identity would violate a core invariant.

S164 Read-your-writes after regional failover [ ] [ ] [ ]

Situation: A user updates a setting in one region, that region fails, and the secondary region has not applied the write yet.

"What can the system promise?"

Answer: The promise depends on synchronous replication and the recovery point objective. If the write was acknowledged only after durable replication to the failover region, read-your-writes can continue at higher latency and lower availability. With asynchronous replication, some acknowledged writes may be missing after regional loss. I would expose a version or receipt so the secondary can wait, recover from a durable log, or tell the user the update is pending rather than showing silent reversal. Product and business owners must choose the data-loss window before failure; the system cannot retroactively provide stronger durability than its acknowledgement policy.

S165 Cross-region replication lag increases [ ] [ ] [ ]

Situation: WAN congestion causes replication lag to grow from seconds to twenty minutes.

"How would you protect users and recovery objectives?"

Answer: I would identify whether bandwidth, packet loss, write volume, or apply capacity is limiting replication, then prioritize critical logs and pause nonessential bulk transfers. Read routing should avoid lagging replicas for operations requiring freshness, and failover may be unsafe if it would exceed the recovery point objective. I would alert on both lag time and bytes remaining, because a quiet period can make time lag look misleading. If sustained traffic exceeds replication capacity, the long-term fix is more bandwidth, better compression or batching, reduced write amplification, or a different regional data placement strategy.

S166 Data must remain within one country [ ] [ ] [ ]

Situation: A new market requires personal data to stay within national boundaries while the product uses global services.

"How would you design for data residency?"

Answer: I would classify which fields and derived data are regulated, then route those users to an approved regional data plane with local databases, object storage, backups, logs, support access, and processing. Global control-plane metadata should contain only permitted identifiers and routing information. Analytics, search, notifications, and disaster recovery must follow the same boundary; storing the primary record locally while exporting raw events would still violate the requirement. I would use policy-aware infrastructure, audited access, and tests that trace data lineage. Residency changes architecture and operations, not just the database region setting.

S167 Payment writes split across regions [ ] [ ] [ ]

Situation: During a partition, two active regions both accept withdrawals from the same balance.

"How would you prevent overspending?"

Answer: A balance invariant cannot safely accept independent conflicting writes and reconcile later. I would assign each account to one authoritative region or use a globally consistent ledger that serializes debits for that account. Other regions may accept an intent but must wait for authorization before confirming success. During loss of the owner, the system may become unavailable for withdrawals rather than risk creating money. Credits or informational reads can have different policies. An immutable ledger, idempotency keys, fencing during failover, and reconciliation protect the boundary, but the core decision is choosing consistency over partition-time write availability.

S168 Evacuating a region overloads the backup [ ] [ ] [ ]

Situation: Traffic shifts away from a failing region, but the secondary reaches saturation and begins failing too.

"How would you avoid cascading regional failure?"

Answer: I would shift traffic gradually while monitoring secondary concurrency, database connections, cache hit rate, queue depth, and dependency quotas. Noncritical features should degrade or shed load before core operations fail. Reserved failover headroom and pre-warmed caches are necessary; an active secondary running at normal peak capacity cannot absorb another region. I would prioritize local users and critical writes according to policy rather than sending all traffic blindly. Global routing uses capacity-aware weights and hysteresis. Regular evacuation drills should measure how much traffic the backup can actually sustain, not assume theoretical instance limits.

S169 Moving users to a new home region [ ] [ ] [ ]

Situation: To reduce latency, millions of user accounts must migrate from one regional database to another.

"How would you migrate without losing writes?"

Answer: I would migrate in cohorts using a versioned routing directory. For each cohort, take a consistent snapshot or bulk copy, capture ongoing changes through a log, and validate counts and checksums while the old region remains authoritative. At cutover, briefly fence or queue writes, apply the remaining delta, change the routing version atomically, and monitor. Requests carrying an old version should be redirected safely rather than writing to both homes. I would retain rollback data and test one small cohort first. Dual writes without ordering and repair rules can create divergence that is harder than the original move.

S170 Global rate limit across regions [ ] [ ] [ ]

Situation: An API key should allow 1,000 requests per minute worldwide, but traffic enters through five regions.

"How would you enforce the limit without adding a global round trip to every request?"

Answer: I would decide whether the limit must be exact. For abuse protection, approximate regional budgets can allocate portions of the global quota and periodically rebalance based on demand. Local token buckets stay fast, while a coordinator adjusts allocations and detects overshoot. A strict financial or contractual quota may require a globally consistent counter or centralized owner, accepting latency and availability cost. I would define burst behavior, fail-open versus fail-closed policy, and what happens when regions cannot coordinate. Metrics should report both local decisions and estimated global consumption so approximation error is visible.

18. Cost, Efficiency, and Capacity Economics

S171 Cloud cost doubles without traffic growth [ ] [ ] [ ]

Situation: Monthly infrastructure cost doubles even though request volume is flat.

"How would you investigate the increase?"

Answer: I would break cost down by service, account, region, resource type, tag, and unit such as cost per request or stored gigabyte. Then I would compare the change date with deployments, autoscaling minimums, retention changes, data transfer, logging, replicas, and idle resources. A flat request count can still hide larger payloads or more work per request. I would stop obvious waste only after confirming ownership and safety. Long term, cost dashboards need allocation tags, anomaly alerts, and unit economics so efficiency regressions appear beside latency and reliability, not weeks later on an invoice.

S172 Cache costs more than the database savings [ ] [ ] [ ]

Situation: A large distributed cache has a high hit rate but costs more than the database capacity it replaced.

"Would you remove it?"

Answer: I would compare total cost and risk, not hit rate alone. The cache may still reduce latency, protect the database during bursts, or avoid expensive downstream work. I would identify low-value keys, oversized values, duplicate copies, fragmentation, TTLs, and memory reserved but unused. Compression, smaller objects, eviction policy, tiered local caching, or a smaller cluster may preserve benefits. I would load-test the origin before reducing capacity and add fallback limits. If the cache provides neither necessary latency nor protection at lower cost, removing it simplifies the system, but the decision needs measured failure and performance impact.

S173 Cross-region egress bill is unexpectedly high [ ] [ ] [ ]

Situation: Services exchange large internal payloads across regions, creating a major network bill.

"How would you reduce egress without harming resilience?"

Answer: I would use flow and billing data to identify top service pairs, payload types, and transfer direction. Then I would colocate chatty components, send compact events instead of full objects, batch or compress transfers, cache immutable data locally, and avoid repeated reads across regions. Replication and backups remain necessary, so I would not remove redundancy blindly. The architecture should distinguish a global control plane from regional data planes and keep user requests local where possible. I would measure cost per operation and verify that optimizations do not create stale authorization data or weaken disaster recovery.

S174 Team wants zero spare capacity [ ] [ ] [ ]

Situation: Finance asks the platform team to run every service near 90 percent utilization to reduce cost.

"How would you explain the reliability risk?"

Answer: High average utilization leaves little room for bursts, failures, deployments, cache misses, or regional evacuation. Queueing delay rises sharply as saturated resources approach their limit, so a small traffic increase can create a large latency increase. I would show load-test curves and define headroom from the service-level objective, scaling speed, and failure model rather than choosing one global percentage. Predictable batch workloads can run hotter than latency-sensitive APIs. Reserved capacity is not automatically waste; it buys response time. Cost optimization should target idle misconfiguration and inefficient work before removing the safety margin required for recovery.

S175 Using spot instances for queue workers [ ] [ ] [ ]

Situation: The team wants cheaper interruptible instances for image-processing workers.

"When is this safe?"

Answer: It is safe when jobs are durably queued, idempotent, checkpointable or short enough to retry, and acknowledgement occurs only after durable completion. Workers must handle termination notice by stopping new work and returning unfinished jobs. I would mix capacity types or providers so one interruption event does not remove the entire fleet and keep minimum reliable capacity for deadlines. Autoscaling should consider queue age and interruption rate. Spot capacity is a poor fit for unique in-memory state or work with expensive nonrepeatable side effects. Savings must include extra retries, slower completion, and operational complexity.

S176 Storage tiering saves money but slows recovery [ ] [ ] [ ]

Situation: Moving backups to a deep archive tier greatly reduces cost but may take hours to restore.

"How would you decide the right storage tier?"

Answer: I would compare retrieval delay and throughput with the recovery time objective, and backup age with the recovery point objective. Frequently needed recent backups can remain on faster storage while older versions move to deep archive. A disaster plan may require a warm replica plus archived backups for different failure types. I would include retrieval fees, parallel restore limits, encryption keys, and provider-region failures in the test. The cheapest monthly storage is unacceptable if recovery cannot meet the business promise. Regular restore drills should measure the complete application recovery, not only archive download time.

S177 Observability bill grows faster than application bill [ ] [ ] [ ]

Situation: Logs, traces, and metrics now cost more than several production services combined.

"How would you reduce cost without becoming blind?"

Answer: I would classify telemetry by operational value. Metrics retain bounded high-level signals, traces use head or tail sampling while preserving errors and slow requests, and logs keep structured security and diagnostic events without repetitive success noise. High-cardinality fields move from metrics to sampled traces or indexed logs with controlled retention. I would remove duplicate ingestion and expensive unused indexes, then measure query usage before shortening retention. Cost budgets and instrumentation review belong in development. The goal is not maximum telemetry volume; it is enough evidence to detect impact, isolate cause, audit risk, and learn from incidents.

S178 Expensive customer queries threaten shared capacity [ ] [ ] [ ]

Situation: A reporting API lets users run filters that trigger full-table scans and large exports.

"How would you add cost guardrails?"

Answer: I would estimate query cost before execution using allowed filters, date range, expected rows, and database plans. Interactive endpoints get strict limits and pagination, while large exports become asynchronous jobs on isolated replicas or warehouses with tenant quotas. I would cancel runaway queries, cap concurrency, and return clear guidance rather than letting one report exhaust the transactional database. Common reports can be precomputed. Audit metrics should show cost by tenant and query shape. Guardrails must be part of the API contract, because scaling the database does not make arbitrary unbounded work safe.

S179 Queue workers scale too aggressively [ ] [ ] [ ]

Situation: Queue depth rises, autoscaling launches hundreds of workers, and the database becomes overloaded.

"How would you make queue-based autoscaling dependency-aware?"

Answer: Queue depth is only one signal; consumers must not exceed the safe concurrency of their slowest dependency. I would scale from oldest-message age and processing rate while enforcing a global database or API concurrency limit. New workers ramp gradually, and the controller observes whether throughput actually improves before adding more. Backpressure, producer limits, and priority queues protect critical work. I would calculate the sustainable service rate and maximum useful workers through load tests. An autoscaler that removes the queue bottleneck by moving it to the database has not increased system capacity.

S180 Build or buy a managed service [ ] [ ] [ ]

Situation: The team must choose between operating its own search cluster and using a managed search service.

"How would you make the decision?"

Answer: I would compare required features, scale, compliance, latency, integration, failure control, portability, staff expertise, on-call burden, and total cost, including engineers and downtime. A managed service accelerates delivery and handles routine operations but introduces vendor limits, pricing, and dependency risk. Self-hosting offers control but requires upgrades, backups, security, scaling, and incident ownership. I would run a representative proof with data volume, queries, recovery, and cost projections. The decision should include an exit or migration plan, but avoiding all lock-in is not free and can delay the product unnecessarily.

19. Architecture Evolution and Migrations

S181 Splitting a growing monolith [ ] [ ] [ ]

Situation: A monolith deploys slowly and several teams block each other, so leadership asks for microservices.

"Where would you start instead of splitting by technical layers?"

Answer: I would identify a business capability with clear ownership, high change pressure, and a boundary that can own its data and API. I would measure whether deployment coupling, scaling, or reliability truly improves by extracting it. Splitting controllers, services, and repositories into separate network services preserves the same coupling and adds latency. The first extraction should use a stable contract, independent deployment, observability, and a rollback path, while the monolith remains functional. Microservices trade code coupling for operational and data consistency complexity, so I would keep modules together when that trade does not buy a clear outcome.

S182 Microservices share one database schema [ ] [ ] [ ]

Situation: Five services deploy independently but directly read and write the same tables.

"What problems does the shared schema create?"

Answer: Independent deployment is weakened because any service can depend on another service's internal columns, locks, and migration timing. A change can break unknown readers, and ownership during incidents is unclear. I would assign table ownership, route new access through the owning service or published events, and use expand-and-contract migrations while removing direct dependencies gradually. A shared database may be acceptable early, but boundaries should still be enforced in code and permissions. Moving every table immediately would create risk; I would start with the highest-coupling writes and add contract tests and access auditing.

S183 Strangler migration from legacy API [ ] [ ] [ ]

Situation: A critical legacy service must be replaced without a single high-risk cutover.

"How would you use the strangler pattern?"

Answer: I would place stable routing in front of the legacy system and move one well-bounded capability at a time to a new implementation. Requests for migrated cases go to the new service, while the rest continue to legacy. I would compare shadow responses where safe, migrate data through a controlled ownership plan, and keep rollback routing until metrics and reconciliation pass. The new service should not call back into legacy for every operation or the old coupling remains. Each slice needs success criteria, observability, and a retirement step so temporary duplication does not become permanent.

S184 Long synchronous service chain [ ] [ ] [ ]

Situation: One user request calls eight services in sequence, and reliability falls as the chain grows.

"How would you shorten the critical path?"

Answer: I would trace the request and classify each call as required for the immediate response, parallelizable, cacheable, precomputable, or asynchronous. Independent calls can run in parallel within a shared deadline, while analytics, notifications, and indexing move to events. Repeated reference data can be cached or embedded in a purpose-built read model. I would avoid simply increasing timeouts because availability multiplies across required dependencies. The remaining synchronous chain gets bounded timeouts, fallbacks where correct, and bulkheads. Fewer network hops also reduce version coupling and make ownership during failures clearer.

S185 Event-driven system is difficult to debug [ ] [ ] [ ]

Situation: A user action triggers twelve asynchronous consumers, and support cannot explain why one outcome is missing.

"How would you make the workflow observable?"

Answer: I would propagate a correlation or causation ID from the original command through every event, retry, and derived event. Each consumer records received, started, completed, failed, and dead-letter states with event ID, schema version, and attempt count. A workflow view can reconstruct the causal graph without relying on log order. Business states such as pending, complete, and failed should be queryable directly. I would avoid pretending asynchronous work is one distributed transaction; compensation and replay remain explicit. Operational tools must respect privacy and not expose full sensitive payloads merely for convenience.

S186 Dual writes during migration diverge [ ] [ ] [ ]

Situation: An application writes both the old and new databases, and occasional failures leave different values in each.

"How would you migrate without unreliable dual writes?"

Answer: I would keep one authoritative write and replicate changes to the new store through a durable change log or outbox. The new store is backfilled from a snapshot, then catches up from a recorded boundary. Reads can be shadowed and compared before gradually switching. Every replicated change is idempotent and versioned, and reconciliation reports differences. If temporary dual writes are unavoidable, failures need durable repair records rather than best-effort logging. At cutover, ownership changes once, with fencing that prevents both stores from independently accepting authoritative writes. Rollback preserves the change stream and known boundary.

S187 API gateway owns business logic [ ] [ ] [ ]

Situation: Authentication, pricing, product rules, response composition, and database calls have accumulated in the API gateway.

"Why is this risky and how would you unwind it?"

Answer: A gateway should handle cross-cutting edge concerns, but business logic there creates a shared deployment bottleneck and unclear domain ownership. Failure or release risk affects every service, and rules may diverge from backend behavior. I would identify logic by owning capability, add or strengthen service APIs, and move rules incrementally while keeping gateway routing compatible. Authentication, coarse rate limiting, request size, and protocol translation may remain. Response aggregation can stay when it is a thin composition layer with bounded calls. Tests should confirm behavior before removing each gateway branch, because a big-bang rewrite would recreate the same risk.

S188 Microservices form a distributed monolith [ ] [ ] [ ]

Situation: Thirty services must be deployed in a fixed order, share releases, and fail together.

"What would you change first?"

Answer: I would map synchronous dependencies, shared databases, incompatible contracts, and coordinated release steps to find the tightest coupling. Services need backward-compatible APIs and events, independent data ownership, and tolerance for mixed versions. I would remove unnecessary synchronous calls, introduce read models or asynchronous integration where appropriate, and combine services that have no meaningful independent boundary. The goal is not preserving service count; it is restoring independent change and failure isolation. Deployment metrics, contract tests, and controlled fault injection show whether a boundary is real. A well-modularized monolith may be better than many inseparable processes.

S189 Replacing a legacy message broker [ ] [ ] [ ]

Situation: The current broker is unsupported, but hundreds of producers and consumers depend on it.

"How would you migrate with low risk?"

Answer: I would inventory topics, schemas, delivery guarantees, ordering, retention, throughput, and consumer ownership before choosing the target. A bridge can copy messages from the old broker to the new one with stable IDs, while selected consumers shadow or migrate gradually. I would compare counts, lag, ordering, and business outcomes and keep consumers idempotent because bridging can duplicate messages. Producers switch only after the new path proves capacity and replay. I would avoid uncontrolled bidirectional bridging, which creates loops. Retirement requires proving no active producers, consumers, or recovery runbooks still depend on the old system.

S190 Breaking service contract is unavoidable [ ] [ ] [ ]

Situation: A service must change a field's meaning, and old consumers cannot interpret the new semantics.

"How would you introduce the change?"

Answer: I would create a new field, endpoint, event version, or media type rather than silently reusing the existing contract. The provider supports both forms during a measured migration window and publishes examples, defaults, and error behavior. Consumer usage is observable so owners can be contacted and completion can be proven. Adapters may translate temporarily, but they need an expiry plan. Contract tests run against known old and new consumers. After usage reaches zero and rollback risk passes, the old contract is removed. Versioning is a cost, but hidden semantic changes create much larger correctness failures.

20. Incident Leadership and Open-Ended Design Decisions

S191 Requirements are vague in a design interview [ ] [ ] [ ]

Situation: An interviewer says, "Design a social-media platform," without giving scale or required features.

"How would you begin?"

Answer: I would narrow the problem before drawing components. I would ask which core actions matter, expected users and peak traffic, media types, latency and availability targets, consistency needs, privacy, and which feature to prioritize. Then I would state reasonable assumptions and estimate read, write, storage, and bandwidth scale. I would design the critical path first, name the source of truth, and expand only where bottlenecks justify it. If time is limited, I would explicitly defer chat, ads, or analytics. Clarifying scope demonstrates judgment; attempting to design every feature produces a shallow list of technologies.

S192 Product asks for five nines everywhere [ ] [ ] [ ]

Situation: A product manager requests 99.999 percent availability for every endpoint without discussing cost or user impact.

"How would you turn that request into an engineering target?"

Answer: I would identify the user journeys that truly require that objective, define success from the user's perspective, and calculate the allowed downtime and error budget. Login, feed browsing, analytics export, and profile decoration do not necessarily need identical targets. Higher availability requires redundancy, simpler dependencies, tested failover, operational staffing, and cost, and may conflict with strong consistency. I would propose tiered service-level objectives and graceful degradation for optional features. The final target should reflect business loss and recovery expectations, not an impressive number applied uniformly without an architecture or budget to support it.

S193 Roll back or fix forward [ ] [ ] [ ]

Situation: A release causes elevated errors, but the team believes a one-line fix can be ready in fifteen minutes.

"How would you choose between rollback and fix forward?"

Answer: I would compare current customer impact, rollback safety and speed, data or schema compatibility, confidence in the proposed fix, and the risk of another deployment. If rollback is known safe and impact is meaningful, I would restore service first rather than keep users waiting for an untested patch. Fix forward is reasonable when rollback would corrupt data, the fault is fully understood, and the change can be validated quickly behind a flag or canary. One incident leader should make the decision with a time bound. Investigation continues after stabilization regardless of which path restores service.

S194 Too many people issue commands during an incident [ ] [ ] [ ]

Situation: Several senior engineers independently change configurations while others investigate, and the system state becomes unclear.

"How would you organize the response?"

Answer: I would establish an incident commander who owns priorities and approves risky changes, an operations lead who executes, investigators for hypotheses, and a communications owner. The team keeps a timestamped log of observations, decisions, commands, results, and current system state. Only one change is made at a time when possible so cause and effect remain visible. A shared channel carries facts; side discussions report back. The commander sets the next checkpoint and stops unproductive work. Clear roles do not require the commander to be the deepest technical expert; they protect coordination while experts focus.

S195 Data corruption may be spreading [ ] [ ] [ ]

Situation: A new worker appears to write malformed records, and downstream systems are copying them.

"What are your first priorities?"

Answer: I would stop or isolate the suspected writer and pause propagation or compaction paths that could overwrite good copies. Preserving evidence and a known-good backup takes priority over immediate cleanup. I would identify the earliest bad version, affected keys, and whether reads can safely filter or route around them. Repair should be scripted, reviewed, idempotent, and tested on a copy; manual edits during uncertainty can spread damage. After containment, I would restore from authoritative sources or reconstruct events, validate checksums and invariants, then resume consumers gradually. Recovery must not destroy the evidence needed to determine scope.

S196 Silent correctness bug has no errors [ ] [ ] [ ]

Situation: A service returns HTTP 200 and normal latency, but a business report shows that some discounts were calculated incorrectly for weeks.

"How would you detect and repair a silent failure?"

Answer: Technical health metrics cannot detect every wrong answer. I would define business invariants and outcome metrics, such as expected price ranges, reconciliation totals, and comparison against an independent calculation. To investigate, I would identify the affected rule version, inputs, time range, and customer cohort, then stop further incorrect decisions through a flag or rollback. Repair may require recalculation, refunds, and clear communication, with an immutable audit trail. I would add rule-version logging, sampled shadow evaluation, and canary comparisons. Success means correct outcomes, not merely successful requests and healthy servers.

S197 Third-party provider outage has no ETA [ ] [ ] [ ]

Situation: A critical external identity provider is down and cannot provide a recovery time.

"How would you operate and communicate during the outage?"

Answer: I would confirm the provider failure, open the escalation channel, and stop retry storms with circuit breakers and bounded backoff. Existing authenticated sessions may continue if locally verifiable and policy permits, while new login or recovery clearly reports degraded service. I would activate any tested secondary provider or manual path only if identity and security semantics remain correct. Communications should state user impact, available workarounds, and the next update time without inventing an ETA. I would monitor provider recovery and ramp traffic gradually. Afterward, I would review dependency concentration, contract terms, and tested alternatives.

S198 Writing a useful post-incident review [ ] [ ] [ ]

Situation: Service has recovered after a major outage, and leadership asks for a postmortem.

"What should the review contain?"

Answer: I would build a factual timeline from detection through recovery, quantify user and business impact, explain the technical and organizational contributing conditions, and identify why existing safeguards did not prevent or limit the event. The review separates root causes from triggers and avoids blaming individuals for reasonable actions in a flawed system. Action items need owners, priorities, and due dates and should cover prevention, detection, containment, and recovery. I would record what worked as well. The document should be understandable to someone outside the incident and later verify whether promised improvements actually reduced risk.

S199 Go or no-go before a major launch [ ] [ ] [ ]

Situation: A launch is scheduled tomorrow, but load testing found a database bottleneck only 20 percent above expected peak traffic.

"Would you launch?"

Answer: I would not decide from the 20 percent number alone. I would review forecast uncertainty, burst shape, scaling time, failure behavior, rollback or feature-disable options, database headroom, and which features can degrade. If expected traffic can plausibly exceed the tested limit and no safe containment exists, I would delay or reduce rollout. A staged launch with strict guardrails, pre-provisioned capacity, and a kill switch may be acceptable. The decision and risk owner should be explicit. Launch pressure does not change queueing behavior, and an avoidable overload is harder to justify after the fact.

S200 Summarizing a complete system design [ ] [ ] [ ]

Situation: You have two minutes left after discussing an Instagram-like architecture in detail.

"How would you close the interview answer clearly?"

Answer: I would summarize the critical user flows, authoritative stores, and the main scaling choices rather than repeat every component. Post creation durably stores metadata and media, then publishes reliable events. Feed reads use hybrid fan-out, caches, cursor pagination, batched hydration, ranking, and CDN media. Idempotency, timeouts, backpressure, and graceful degradation protect failures, while metrics, traces, replay, backups, and tested failover support operations. I would restate the deliberate trade-offs: eventual consistency for feeds and counters, stronger consistency for identity and permissions, and added asynchronous complexity in exchange for lower latency and better isolation.

60-SECOND ANSWER SHAPE

1. State the leading hypothesis without claiming certainty. **2.** Name the first metrics, traces, or logs to compare. **3.** Contain customer impact before deep investigation. **4.** Isolate the bottleneck or failed boundary. **5.** Finish with the durable fix, trade-off, and prevention signal.

MORNING SCENARIO REVIEW LOG

Record the scenarios you attempted and the part of your answer that needs repair before the next recall.

Date	Drill IDs	Weak spot / repair	/5